

Second Bloom 旧衣新生系统软件著作权源程序-普通交存

软件版本：V1.0 交存方式：前 30 页 + 后 30 页 每页 50 行连续页码

```
00001 // File: app/src/main/java/com/scf/secondbloom/MainActivity.kt
00002 package com.scf.secondbloom
00003
00004 import android.os.Bundle
00005 import androidx.activity.ComponentActivity
00006 import androidx.activity.compose.setContent
00007 import androidx.activity.enableEdgeToEdge
00008 import com.clerk.api.Clerk
00009 import com.scf.secondbloom.auth.SecondBloomClerkConfig
00010 import com.scf.secondbloom.ui.MainScreen
00011 import com.scf.secondbloom.ui.theme.SecondBloomTheme
00012
00013 class MainActivity : ComponentActivity() {
00014     override fun onCreate(savedInstanceState: Bundle?) {
00015         super.onCreate(savedInstanceState)
00016         if (SecondBloomClerkConfig.isConfigured) {
00017             Clerk.auth.handle(intent.data)
00018         }
00019         enableEdgeToEdge()
00020         setContent {
00021             SecondBloomTheme {
00022                 MainScreen()
00023             }
00024         }
00025     }
00026
00027     override fun onNewIntent(intent: android.content.Intent) {
00028         super.onNewIntent(intent)
00029         setIntent(intent)
00030         if (SecondBloomClerkConfig.isConfigured) {
00031             Clerk.auth.handle(intent.data)
00032         }
00033     }
00034 }
00035 // File: app/src/main/java/com/scf/secondbloom/ui/MainScreen.kt
00036 package com.scf.secondbloom.ui
00037
00038 import androidx.compose.animation.AnimatedContentTransitionScope
00039 import androidx.compose.animation.core.FastOutSlowInEasing
00040 import androidx.compose.animation.core.tween
00041 import androidx.compose.foundation.layout.WindowInsets
00042 import androidx.compose.foundation.layout.fillMaxSize
00043 import androidx.compose.foundation.layout.padding
00044 import androidx.compose.material3.MaterialTheme
00045 import androidx.compose.material3.Scaffold
00046 import androidx.compose.runtime.Composable
00047 import androidx.compose.runtime.CompositionLocalProvider
00048 import androidx.compose.runtime.LaunchedEffect
00049 import androidx.compose.runtime.getValue
00050 import androidx.compose.runtime.mutableStateOf
```

```

00051 import androidx.compose.runtime.rememberCoroutineScope
00052 import androidx.compose.runtime.setValue
00053 import androidx.compose.runtime.saveable.rememberSaveable
00054 import androidx.compose.ui.Modifier
00055 import androidx.lifecycle.compose.collectAsStateWithLifecycle
00056 import androidx.lifecycle.viewmodel.compose.viewModel
00057 import androidx.navigation.NavHostController
00058 import androidx.navigation.NavGraph.Companion.findStartDestination
00059 import androidx.navigation.NavType
00060 import androidx.navigation.compose.NavHost
00061 import androidx.navigation.compose.composable
00062 import androidx.navigation.compose.currentBackStackEntryAsState
00063 import androidx.navigation.compose.rememberNavController
00064 import androidx.navigation.navArgument
00065 import com.scf.secondbloom.auth.rememberSecondBloomAuthUiState
00066 import com.scf.secondbloom.auth.signOutCurrentUser
00067 import com.scf.secondbloom.navigation.Screen
00068 import com.scf.secondbloom.presentation remodel.RemodelViewModel
00069 import com.scf.secondbloom.ui.components.SecondBloomBottomNavBar
00070 import com.scf.secondbloom.ui.i18n.LocalAppLanguage
00071 import com.scf.secondbloom.ui.screens.AuthScreen
00072 import com.scf.secondbloom.ui.screens.AccountScreen
00073 import com.scf.secondbloom.ui.screens.HomeScreen
00074 import com.scf.secondbloom.ui.screens.InspirationDetailScreen
00075 import com.scf.secondbloom.ui.screens.InspirationScreen
00076 import com.scf.secondbloom.ui.screens.PlanetScreen
00077 import com.scf.secondbloom.ui.screens.PreviewEditorScreen
00078 import com.scf.secondbloom.ui.screens.PreviewResultScreen
00079 import com.scf.secondbloom.ui.screens.ProfileScreen
00080 import com.scf.secondbloom.ui.screens.WardrobeScreen
00081 import com.scf.secondbloom.ui.screens.WorkbenchScreen
00082 import kotlinx.coroutines.launch
00083
00084 private const val AnimationDuration = 300
00085
00086 @Composable
00087 fun MainScreen(
00088     modifier: Modifier = Modifier,
00089     navController: NavHostController = rememberNavController(),
00090     remodelViewModel: RemodelViewModel = viewModel(factory = RemodelViewModel.Factory)
00091 ) {
00092     val navBackStackEntry by navController.currentBackStackEntryAsState()
00093     val currentRoute = navBackStackEntry?.destination?.route
00094     val uiState by remodelViewModel.uiState.collectAsStateWithLifecycle()
00095     val authState = rememberSecondBloomAuthUiState()
00096     val coroutineScope = rememberCoroutineScope()
00097     var highlightedTopLevelRoute by rememberSaveable
    { mutableStateOf(Screen.Inspiration.route) }
00098
00099     LaunchedEffect(currentRoute) {
00100         if (currentRoute in Screen.topLevelItems().map { it.route }) {

```

```

00101         highlightedTopLevelRoute = currentRoute ?: Screen.Inspiration.route
00102     }
00103 }
00104
00105 CompositionLocalProvider(LocalAppLanguage provides uiState.appLanguage) {
00106     Scaffold(
00107         modifier = modifier.fillMaxSize(),
00108         containerColor = MaterialTheme.colorScheme.background,
00109         contentColor = MaterialTheme.colorScheme.onBackground,
00110         contentWindowInsets = WindowInsets(0, 0, 0, 0),
00111         bottomBar = {
00112             if (currentRoute in Screen.topLevelItems().map { it.route }) {
00113                 SecondBloomBottomNavBar(
00114                     highlightedRoute = highlightedTopLevelRoute,
00115                     onNavigate = { route ->
00116                         if (currentRoute != route) {
00117                             val restoredExistingDestination = navController.popBackStack(
00118                                 route = route,
00119                                 inclusive = false
00120                             )
00121                             if (!restoredExistingDestination) {
00122                                 navController.navigate(route) {
00123                                     popUpTo(navController.graph.findStartDestination().id) {
00124                                         saveState = true
00125                                     }
00126                                     launchSingleTop = true
00127                                     restoreState = true
00128                                 }
00129                             }
00130                         }
00131                     },
00132                     onOpenRemodelFlow = {
00133                         if (currentRoute != Screen.CameraRecognition.route) {
00134                             navController.navigate(Screen.CameraRecognition.route) {
00135                                 launchSingleTop = true
00136                             }
00137                         }
00138                     }
00139                 )
00140             }
00141         }
00142     ) { innerPadding ->
00143         NavHost(
00144             navController = navController,
00145             startDestination = Screen.Inspiration.route,
00146             modifier = Modifier
00147                 .fillMaxSize()
00148                 .padding(innerPadding)
00149         ) {
00150             composable(

```

```

00151         route = Screen.Inspiration.route,
00152         enterTransition = { slideInForward() },
00153         exitTransition = { slideOutForward() },
00154         popEnterTransition = { slideInBack() },
00155         popExitTransition = { slideOutBack() }
00156     ) {
00157         InspirationScreen(
00158             state = uiState,
00159             onOpenInspirationDetail = { itemId ->
00160                 navController.navigate(Screen.InspirationDetail.createRoute(itemId))
00161             },
00162             launchSingleTop = true
00163         ),
00164         onOpenRemodelFlow = {
00165             navController.navigate(Screen.CameraRecognition.route) {
00166                 launchSingleTop = true
00167             }
00168         }
00169     )
00170 }
00171
00172 composable(
00173     route = Screen.InspirationDetail.route,
00174     arguments = listOf(
00175         navArgument(Screen.InspirationDetail.itemIdArg) {
00176             type = NavType.StringType
00177         }
00178     ),
00179     enterTransition = { slideInForward() },
00180     exitTransition = { slideOutForward() },
00181     popEnterTransition = { slideInBack() },
00182     popExitTransition = { slideOutBack() }
00183 ) { backStackEntry ->
00184     val itemId =
00185     backStackEntry.arguments?.getString(Screen.InspirationDetail.itemIdArg)
00186     val item = itemId?.let {
00187         com.scf.secondbloom.ui.model.SecondBloomShowcaseContent.inspirationCardById(
00188             itemId = it,
00189             records = uiState.publishedRemodelRecords,
00190             engagements = uiState.inspirationEngagementRecords,
00191             language = uiState.appLanguage
00192         )
00193     }
00194     InspirationDetailScreen(
00195         state = uiState,
00196         item = item,
00197         onBack = { navController.popBackStack() },
00198         onToggleLike = remodelViewModel::toggleInspirationLike,
00199         onToggleBookmark = remodelViewModel::toggleInspirationBookmark,
00200         onAddComment = remodelViewModel::addInspirationComment,
00201         onOpenPublishedResult = { planId ->

```

```

00201         navController.navigate(Screen.PreviewResult.createRoute(planId)) {
00202             launchSingleTop = true
00203         }
00204     },
00205     onOpenRemodelFlow = {
00206         navController.navigate(Screen.CameraRecognition.route) {
00207             launchSingleTop = true
00208         }
00209     }
00210 )
00211 }
00212
00213 composable(
00214     route = Screen.Wardrobe.route,
00215     enterTransition = { slideInForward() },
00216     exitTransition = { slideOutForward() },
00217     popEnterTransition = { slideInBack() },
00218     popExitTransition = { slideOutBack() }
00219 ) {
00220     WardrobeScreen(
00221         state = uiState,
00222         onOpenRemodelFlow = {
00223             navController.navigate(Screen.CameraRecognition.route) {
00224                 launchSingleTop = true
00225             }
00226         },
00227         onOpenSavedPlan = { recordId ->
00228             val restored = remodelViewModel.restoreSavedPlanGeneration(recordId)
00229             if (restored) {
00230                 navController.navigate(Screen.Plan.route) {
00231                     launchSingleTop = true
00232                 }
00233             }
00234         }
00235     )
00236 }
00237
00238 composable(
00239     route = Screen.Planet.route,
00240     enterTransition = { slideInForward() },
00241     exitTransition = { slideOutForward() },
00242     popEnterTransition = { slideInBack() },
00243     popExitTransition = { slideOutBack() }
00244 ) {
00245     PlanetScreen(
00246         state = uiState,
00247         onOpenRemodelFlow = {
00248             navController.navigate(Screen.CameraRecognition.route) {
00249                 launchSingleTop = true
00250             }

```

```

00251     }
00252   )
00253 }
00254
00255 composable(
00256     route = Screen.Profile.route,
00257     enterTransition = { slideInForward() },
00258     exitTransition = { slideOutForward() },
00259     popEnterTransition = { slideInBack() },
00260     popExitTransition = { slideOutBack() }
00261 ) {
00262     ProfileScreen(
00263         state = uiState,
00264         authState = authState,
00265         onLanguageSelected = remodelViewModel::setAppLanguage,
00266         onLoginClick = {
00267             navController.navigate(Screen.Auth.route) {
00268                 launchSingleTop = true
00269             }
00270         },
00271         onAccountClick = {
00272             navController.navigate(Screen.Account.route) {
00273                 launchSingleTop = true
00274             }
00275         },
00276         onLogoutClick = {
00277             coroutineScope.launch {
00278                 signOutCurrentUser()
00279             }
00280         }
00281     )
00282 }
00283
00284 composable(
00285     route = Screen.Auth.route,
00286     enterTransition = { slideInForward() },
00287     exitTransition = { slideOutForward() },
00288     popEnterTransition = { slideInBack() },
00289     popExitTransition = { slideOutBack() }
00290 ) {
00291     AuthScreen(
00292         onDismiss = { navController.popBackStack() }
00293     )
00294 }
00295
00296 composable(
00297     route = Screen.Account.route,
00298     enterTransition = { slideInForward() },
00299     exitTransition = { slideOutForward() },
00300     popEnterTransition = { slideInBack() },

```

```

00301         popExitTransition = { slideOutBack() }
00302     ) {
00303         AccountScreen(
00304             authState = authState,
00305             onDismiss = { navController.popBackStack() }
00306         )
00307     }
00308
00309     composable(
00310         route = Screen.CameraRecognition.route,
00311         enterTransition = { slideInForward() },
00312         exitTransition = { slideOutForward() },
00313         popEnterTransition = { slideInBack() },
00314         popExitTransition = { slideOutBack() }
00315     ) {
00316         HomeScreen(
00317             state = uiState,
00318             onImageSelected = remodelViewModel::onImageSelected,
00319             onLoadDemoScenario = remodelViewModel::loadDemoScenario,
00320             onAnalyze = remodelViewModel::analyzeSelectedImage,
00321             onContinueLowConfidence = remodelViewModel::continueWithLowConfidence,
00322             onDismissError = remodelViewModel::clearError,
00323             onOpenWorkbench = {
00324                 navController.navigate(Screen.Plan.route) {
00325                     launchSingleTop = true
00326                 }
00327             }
00328         )
00329     }
00330
00331     composable(
00332         route = Screen.Plan.route,
00333         enterTransition = { slideInForward() },
00334         exitTransition = { slideOutForward() },
00335         popEnterTransition = { slideInBack() },
00336         popExitTransition = { slideOutBack() }
00337     ) {
00338         WorkbenchScreen(
00339             state = uiState,
00340             onGarmentTypeChange = remodelViewModel::updateGarmentType,
00341             onColorChange = remodelViewModel::updateColor,
00342             onMaterialChange = remodelViewModel::updateMaterial,
00343             onStyleChange = remodelViewModel::updateStyle,
00344             onDefectsChange = remodelViewModel::updateDefects,
00345             onIntentSelected = remodelViewModel::selectIntent,
00346             onPreferencesChange = remodelViewModel::updateUserPreferences,
00347             onGeneratePlans = remodelViewModel::generatePlans,
00348             onOpenPreviewEditor = { planId ->
00349                 navController.navigate(Screen.PreviewEditor.createRoute(planId)) {
00350                     launchSingleTop = true

```

```

00351         }
00352     },
00353     onOpenPreviewResult = { planId ->
00354         navController.navigate(Screen.PreviewResult.createRoute(planId)) {
00355             launchSingleTop = true
00356         }
00357     },
00358     onDismissError = remodelViewModel::clearError
00359 )
00360 }
00361
00362 composable(
00363     route = Screen.PreviewEditor.route,
00364     arguments = listOf(
00365         navArgument(Screen.PreviewEditor.planIdArg) {
00366             type = NavType.StringType
00367         }
00368     ),
00369     enterTransition = { slideInForward() },
00370     exitTransition = { slideOutForward() },
00371     popEnterTransition = { slideInBack() },
00372     popExitTransition = { slideOutBack() }
00373 ) { backStackEntry ->
00374     val planId =
backStackEntry.arguments?.getString(Screen.PreviewEditor.planIdArg)
00375     PreviewEditorScreen(
00376         state = uiState,
00377         planId = planId,
00378         onBack = { navController.popBackStack() },
00379         onOpenPreviewEditor = remodelViewModel::openPreviewEditor,
00380         onClosePreviewEditor = remodelViewModel::closePreviewEditor,
00381         onSilhouetteChange = remodelViewModel::updatePreviewEditSilhouette,
00382         onLengthChange = remodelViewModel::updatePreviewEditLength,
00383         onNecklineChange = remodelViewModel::updatePreviewEditNeckline,
00384         onSleeveChange = remodelViewModel::updatePreviewEditSleeve,
00385         onFidelityChange = remodelViewModel::updatePreviewEditFidelity,
00386         onInstructionsChange = remodelViewModel::updatePreviewEditInstructions,
00387         onGenerateFinalImage = remodelViewModel::generatePreviewFromEditor,
00388         onOpenPreviewResult = { selectedPlanId ->
00389             navController.navigate(Screen.PreviewResult.createRoute(selectedPlanId)) {
00390                 launchSingleTop = true
00391             }
00392         }
00393     )
00394 }
00395
00396 composable(
00397     route = Screen.PreviewResult.route,
00398     arguments = listOf(
00399         navArgument(Screen.PreviewResult.planIdArg) {
00400             type = NavType.StringType

```



```

00401         }
00402     ),
00403     enterTransition = { slideInForward() },
00404     exitTransition = { slideOutForward() },
00405     popEnterTransition = { slideInBack() },
00406     popExitTransition = { slideOutBack() }
00407 ) { backStackEntry ->
00408     val planId =
backStackEntry.arguments?.getString(Screen.PreviewResult.planIdArg)
00409     PreviewResultScreen(
00410         state = uiState,
00411         planId = planId,
00412         onBack = { navController.popBackStack() },
00413         onBackToPlans = {
00414             navController.navigate(Screen.Plan.route) {
00415                 popUpTo(Screen.Plan.route) {
00416                     inclusive = false
00417                 }
00418                 launchSingleTop = true
00419             }
00420         },
00421         onEditPlan = { selectedPlanId ->
00422             navController.navigate(Screen.PreviewEditor.createRoute(selectedPlanId)) {
00423                 launchSingleTop = true
00424             }
00425         },
00426         onPublish = remodelViewModel::publishPreviewResult,
00427         onOpenInspiration = {
00428             navController.navigate(Screen.Inspiration.route) {
00429                 popUpTo(Screen.Inspiration.route) {
00430                     inclusive = false
00431                 }
00432                 launchSingleTop = true
00433             }
00434         },
00435         onResumePolling = remodelViewModel::resumePreviewPolling,
00436         onDismissError = remodelViewModel::clearError
00437     )
00438 }
00439 }
00440 }
00441 }
00442 }
00443
00444 private fun AnimatedContentTransitionScope<*>.slideInForward() =
00445     slideIntoContainer(
00446         towards = AnimatedContentTransitionScope.SlideDirection.Start,
00447         animationSpec = tween(
00448             durationMillis = AnimationDuration,
00449             easing = FastOutSlowInEasing
00450     )

```

```

00451     )
00452
00453 private fun AnimatedContentTransitionScope<*>.slideOutForward() =
00454     slideOutOfContainer(
00455         towards = AnimatedContentTransitionScope.SlideDirection.Start,
00456         animationSpec = tween(
00457             durationMillis = AnimationDuration,
00458             easing = FastOutSlowInEasing
00459         )
00460     )
00461
00462 private fun AnimatedContentTransitionScope<*>.slideInBack() =
00463     slideIntoContainer(
00464         towards = AnimatedContentTransitionScope.SlideDirection.End,
00465         animationSpec = tween(
00466             durationMillis = AnimationDuration,
00467             easing = FastOutSlowInEasing
00468         )
00469     )
00470
00471 private fun AnimatedContentTransitionScope<*>.slideOutBack() =
00472     slideOutOfContainer(
00473         towards = AnimatedContentTransitionScope.SlideDirection.End,
00474         animationSpec = tween(
00475             durationMillis = AnimationDuration,
00476             easing = FastOutSlowInEasing
00477         )
00478     )
00479 // File: app/src/main/java/com/scf/secondbloom/navigation/Screen.kt
00480 package com.scf.secondbloom.navigation
00481
00482 import androidx.compose.material.icons.Icons
00483 import androidx.compose.material.icons.filled.Checkroom
00484 import androidx.compose.material.icons.filled.AddCircle
00485 import androidx.compose.material.icons.filled.AutoAwesome
00486 import androidx.compose.material.icons.filled.Person
00487 import androidx.compose.material.icons.filled.Public
00488 import androidx.compose.material.icons.outlined.Checkroom
00489 import androidx.compose.material.icons.outlined.AddCircleOutline
00490 import androidx.compose.material.icons.outlined.AutoAwesome
00491 import androidx.compose.material.icons.outlined.Person
00492 import androidx.compose.material.icons.outlined.Public
00493 import androidx.compose.ui.graphics.vector.ImageVector
00494
00495 sealed class Screen(
00496     val route: String,
00497     val title: String,
00498     val selectedIcon: ImageVector,
00499     val unselectedIcon: ImageVector,
00500     val contentDescription: String,

```

```

00501     val showInBottomBar: Boolean
00502   ) {
00503     data object Inspiration : Screen(
00504       route = "inspiration",
00505       title = "灵感空间",
00506       selectedIcon = Icons.Filled.AutoAwesome,
00507       unselectedIcon = Icons.Outlined.AutoAwesome,
00508       contentDescription = "灵感空间页面",
00509       showInBottomBar = true
00510     )
00511
00512     data object InspirationDetail : Screen(
00513       route = "inspiration_detail/{itemId}",
00514       title = "灵感详情",
00515       selectedIcon = Icons.Filled.AutoAwesome,
00516       unselectedIcon = Icons.Outlined.AutoAwesome,
00517       contentDescription = "灵感详情页面",
00518       showInBottomBar = false
00519     ) {
00520       const val itemIdArg = "itemId"
00521
00522       fun createRoute(itemId: String): String = "inspiration_detail/$itemId"
00523     }
00524
00525     data object Wardrobe : Screen(
00526       route = "wardrobe",
00527       title = "数字衣橱",
00528       selectedIcon = Icons.Filled.Checkroom,
00529       unselectedIcon = Icons.Outlined.Checkroom,
00530       contentDescription = "数字衣橱页面",
00531       showInBottomBar = true
00532     )
00533
00534     data object Planet : Screen(
00535       route = "planet",
00536       title = "可持续星球",
00537       selectedIcon = Icons.Filled.Public,
00538       unselectedIcon = Icons.Outlined.Public,
00539       contentDescription = "可持续星球页面",
00540       showInBottomBar = true
00541     )
00542
00543     data object Profile : Screen(
00544       route = "profile",
00545       title = "我的主页",
00546       selectedIcon = Icons.Filled.Person,
00547       unselectedIcon = Icons.Outlined.Person,
00548       contentDescription = "我的主页页面",
00549       showInBottomBar = true
00550     )

```

```

00551
00552 data object Auth : Screen(
00553     route = "auth",
00554     title = "登录",
00555     selectedIcon = Icons.Filled.Person,
00556     unselectedIcon = Icons.Outlined.Person,
00557     contentDescription = "登录页面",
00558     showInBottomBar = false
00559 )
00560
00561 data object Account : Screen(
00562     route = "account",
00563     title = "账号设置",
00564     selectedIcon = Icons.Filled.Person,
00565     unselectedIcon = Icons.Outlined.Person,
00566     contentDescription = "账号设置页面",
00567     showInBottomBar = false
00568 )
00569
00570 data object CameraRecognition : Screen(
00571     route = "camera_recognition",
00572     title = "上传识别",
00573     selectedIcon = Icons.Filled.AddCircle,
00574     unselectedIcon = Icons.Outlined.AddCircleOutline,
00575     contentDescription = "上传识别页面",
00576     showInBottomBar = false
00577 )
00578
00579 data object Plan : Screen(
00580     route = "plan",
00581     title = "制衣方案",
00582     selectedIcon = Icons.Filled.AutoAwesome,
00583     unselectedIcon = Icons.Outlined.AutoAwesome,
00584     contentDescription = "制衣方案页面",
00585     showInBottomBar = false
00586 )
00587
00588 data object PreviewEditor : Screen(
00589     route = "preview_editor/{planId}",
00590     title = "真图编辑",
00591     selectedIcon = Icons.Filled.AutoAwesome,
00592     unselectedIcon = Icons.Outlined.AutoAwesome,
00593     contentDescription = "真图编辑页面",
00594     showInBottomBar = false
00595 ) {
00596     const val planIdArg = "planId"
00597
00598     fun createRoute(planId: String): String = "preview_editor/$planId"
00599 }
00600

```

```

00601 data object PreviewResult : Screen(
00602     route = "preview_result/{planId}",
00603     title = "最终效果图",
00604     selectedIcon = Icons.Filled.AutoAwesome,
00605     unselectedIcon = Icons.Outlined.AutoAwesome,
00606     contentDescription = "最终效果图页面",
00607     showInBottomBar = false
00608 ) {
00609     const val planIdArg = "planId"
00610
00611     fun createRoute(planId: String): String = "preview_result/$planId"
00612 }
00613
00614 companion object {
00615     fun topLevelItems(): List<Screen> = listOf(Inspiration, Wardrobe, Planet, Profile)
00616
00617     fun flowItems(): List<Screen> = listOf(
00618         InspirationDetail,
00619         CameraRecognition,
00620         Plan,
00621         PreviewEditor,
00622         PreviewResult,
00623         Auth,
00624         Account
00625     )
00626
00627     fun allItems(): List<Screen> = topLevelItems() + flowItems()
00628 }
00629 }
00630 // File: app/src/main/java/com/scf/secondbloom/presentation/remodel/RemodelViewModel.kt
00631 package com.scf.secondbloom.presentation.remodel
00632
00633 import android.app.Application
00634 import androidx.lifecycle.ViewModel
00635 import androidx.lifecycle.ViewModelProvider
00636 import androidx.lifecycle.viewModelScope
00637 import com.clerk.api.Clerk
00638 import com.scf.secondbloom.BuildConfig
00639 import com.scf.secondbloom.auth.ClerkHistoryAuthTokenProvider
00640 import com.scf.secondbloom.auth.SecondBloomClerkConfig
00641 import com.scf.secondbloom.data.local.AppPreferencesRepository
00642 import com.scf.secondbloom.data.local.AppPreferencesRepositoryFactory
00643 import com.scf.secondbloom.data.local.NoOpRemodelHistoryRepository
00644 import com.scf.secondbloom.data.local.RemodelHistoryRepository
00645 import com.scf.secondbloom.data.local.RemodelHistoryRepositoryFactory
00646 import com.scf.secondbloom.data.local.RemodelHistorySyncRepositoryFactory
00647 import com.scf.secondbloom.data.remote.InvalidImageException
00648 import com.scf.secondbloom.data.remote.ModelResponseException
00649 import com.scf.secondbloom.data.remote.mock.MockRemodelApi
00650 import com.scf.secondbloom.data.repository.DefaultRemodelRepository

```

```
00651 import com.scf.secondbloom.data.repository.RemodelRepository
00652 import com.scf.secondbloom.data.repository.RemodelRepositoryFactory
00653 import com.scf.secondbloom.data.historysync.HistorySyncRepository
00654 import com.scf.secondbloom.data.historysync.HistorySyncResult
00655 import com.scf.secondbloom.domain.model.AppLanguage
00656 import com.scf.secondbloom.domain.model.BackgroundComplexity
00657 import com.scf.secondbloom.domain.model.DemoScenario
00658 import com.scf.secondbloom.domain.model.GarmentAnalysis
00659 import com.scf.secondbloom.domain.model.GarmentDefect
00660 import com.scf.secondbloom.domain.model.InspirationComment
00661 import com.scf.secondbloom.domain.model.InspirationEngagementRecord
00662 import com.scf.secondbloom.domain.model.PreviewEditFidelity
00663 import com.scf.secondbloom.domain.model.PreviewEditLength
00664 import com.scf.secondbloom.domain.model.PreviewEditNeckline
00665 import com.scf.secondbloom.domain.model.PreviewEditOptions
00666 import com.scf.secondbloom.domain.model.PreviewEditSilhouette
00667 import com.scf.secondbloom.domain.model.PreviewEditSleeve
00668 import com.scf.secondbloom.domain.model.PlanPreviewResult
00669 import com.scf.secondbloom.domain.model.PreviewJobSnapshot
00670 import com.scf.secondbloom.domain.model.PreviewJobStatus
00671 import com.scf.secondbloom.domain.model.PreviewRenderStatus
00672 import com.scf.secondbloom.domain.model.RemodelError
00673 import com.scf.secondbloom.domain.model.RemodelErrorType
00674 import com.scf.secondbloom.domain.model.RemodelIntent
00675 import com.scf.secondbloom.domain.model.RemodelStage
00676 import com.scf.secondbloom.domain.model.RemodelUiState
00677 import com.scf.secondbloom.domain.model.RemodelPlan
00678 import com.scf.secondbloom.domain.model.SelectedImage
00679 import com.scf.secondbloom.domain.model.deriveRecentActivities
00680 import com.scf.secondbloom.domain.model.deriveSustainabilityImpactSummary
00681 import com.scf.secondbloom.domain.model.deriveWardrobeCategories
00682 import com.scf.secondbloom.domain.model.deriveWardrobeEntries
00683 import java.io.IOException
00684 import java.util.UUID
00685 import kotlinx.coroutines.delay
00686 import kotlinx.coroutines.flow.Flow
00687 import kotlinx.coroutines.flow.MutableStateFlow
00688 import kotlinx.coroutines.flow.StateFlow
00689 import kotlinx.coroutines.flow.asStateFlow
00690 import kotlinx.coroutines.flow.flowOf
00691 import kotlinx.coroutines.flow.map
00692 import kotlinx.coroutines.flow.update
00693 import kotlinx.coroutines.launch
00694 import kotlinx.coroutines.Job
00695
00696 private const val LowConfidenceThreshold = 0.75f
00697 private const val PreviewPollDelayMillis = 1_000L
00698 private const val PreviewMaxPollAttempts = 30
00699 private val ActivePreviewStatuses = setOf(PreviewJobStatus.QUEUED, PreviewJobStatus.RUNNING)
00700 private val ActivePreviewRenderStatuses = setOf(PreviewRenderStatus.QUEUED,
PreviewRenderStatus.RUNNING)
```

```

00701
00702 class RemodelViewModel(
00703     private val repository: RemodelRepository,
00704     private val demoRepository: RemodelRepository =
DefaultRemodelRepository(MockRemodelApi()),
00705     private val historyRepository: RemodelHistoryRepository = NoOpRemodelHistoryRepository,
00706     private val historySyncRepositoryFactory: ((String) -> HistorySyncRepository)? = null,
00707     private val authUserIdFlow: Flow<String?> = if (SecondBloomClerkConfig.isConfigured) {
00708         Clerk.userFlow.map { user -> user?.id }
00709     } else {
00710         flowOf(null)
00711     },
00712     private val appPreferencesRepository: AppPreferencesRepository = object :
AppPreferencesRepository {
00713         override fun getAppLanguage(): AppLanguage = AppLanguage.ENGLISH
00714
00715         override fun setAppLanguage(language: AppLanguage) = Unit
00716     }
00717 ) : ViewModel() {
00718
00719     private var activeHistorySyncRepository: HistorySyncRepository? = null
00720     private var activeHistorySyncUserId: String? = null
00721     private var activeHistorySyncJob: Job? = null
00722     private var pendingHistorySyncAfterActiveJob: Boolean = false
00723
00724     private val _uiState = MutableStateFlow(
00725         RemodelUiState(appLanguage = appPreferencesRepository.getAppLanguage())
00726     )
00727     val uiState: StateFlow<RemodelUiState> = _uiState.asStateFlow()
00728
00729     init {
00730         viewModelScope.launch {
00731             refreshHistory()
00732         }
00733         observeAuthState()
00734     }
00735
00736     fun onImageSelected(image: SelectedImage) {
00737         _uiState.update { current ->
00738             current.copy(
00739                 stage = RemodelStage.ImageSelected,
00740                 selectedImage = image,
00741                 analysis = null,
00742                 draftAnalysis = null,
00743                 selectedIntent = null,
00744                 userPreferences = "",
00745                 plans = emptyList(),
00746                 selectedPlanId = null,
00747                 editingPlanId = null,
00748                 previewEditOptions = PreviewEditOptions(),
00749                 previewJob = null,
00750                 isPreviewLoading = false,

```

```

00751         previewErrorMessage = null,
00752         error = null
00753     )
00754 }
00755 }
00756
00757 fun setAppLanguage(language: AppLanguage) {
00758     if (_uiState.value.appLanguage == language) {
00759         return
00760     }
00761     appPreferencesRepository.setAppLanguage(language)
00762     _uiState.update { it.copy(appLanguage = language) }
00763     viewModelScope.launch {
00764         refreshHistory()
00765     }
00766 }
00767
00768 fun loadDemoScenario(scenario: DemoScenario) {
00769     onImageSelected(scenario.toSelectedImage())
00770 }
00771
00772 private fun activeRepository(state: RemodelUiState = _uiState.value): RemodelRepository =
00773     if (state.selectedDemoScenario != null) demoRepository else repository
00774
00775 fun analyzeSelectedImage() {
00776     val image = _uiState.value.selectedImage
00777     if (image == null) {
00778         val language = _uiState.value.appLanguage
00779         setError(
00780             stage = RemodelStage.InvalidImage,
00781             error = RemodelError(
00782                 type = RemodelErrorType.INVALID_IMAGE,
00783                 message = tr(
00784                     language,
00785                     "Choose a garment photo before starting analysis.",
00786                     "请先选择一张旧衣图片。"
00787                 )
00788             )
00789         )
00790         return
00791     }
00792
00793     viewModelScope.launch {
00794         val activeRepository = activeRepository()
00795         _uiState.update {
00796             it.copy(
00797                 stage = RemodelStage.Analyzing,
00798                 plans = emptyList(),
00799                 error = null
00800             )

```



```

00801     }
00802
00803     try {
00804         val analysis = activeRepository.analyze(image, _uiState.value.applLanguage)
00805         historyRepository.saveAnalysis(image, analysis)
00806         refreshHistory()
00807         syncHistoryInBackground()
00808
00809         val nextStage = if (
00810             analysis.backgroundComplexity == BackgroundComplexity.HIGH ||
00811             analysis.confidence < LowConfidenceThreshold
00812         ) {
00813             RemodelStage.LowConfidence
00814         } else {
00815             RemodelStage.AnalysisReady
00816         }
00817
00818         _uiState.update {
00819             it.copy(
00820                 stage = nextStage,
00821                 analysis = analysis,
00822                 draftAnalysis = analysis,
00823                 plans = emptyList(),
00824                 error = null
00825             )
00826         }
00827     } catch (exception: InvalidImageException) {
00828         setError(
00829             stage = RemodelStage.InvalidImage,
00830             error = RemodelError(
00831                 RemodelErrorType.INVALID_IMAGE,
00832                 exception.message.orEmpty()
00833             )
00834         )
00835     } catch (exception: IOException) {
00836         setError(
00837             stage = RemodelStage.NetworkError,
00838             error = RemodelError(RemodelErrorType.NETWORK_ERROR,
exception.message.orEmpty())
00839         )
00840     } catch (exception: ModelResponseException) {
00841         setError(
00842             stage = RemodelStage.ModelError,
00843             error = RemodelError(RemodelErrorType.MODEL_ERROR,
exception.message.orEmpty())
00844         )
00845     }
00846 }
00847
00848 fun continueWithLowConfidence() {
00849     val draftAnalysis = _uiState.value.draftAnalysis ?: return
00850

```

```

00851     _uiState.update {
00852         it.copy(
00853             stage = RemodelStage.AnalysisReady,
00854             analysis = draftAnalysis,
00855             error = null
00856         )
00857     }
00858 }
00859
00860 fun clearError() {
00861     _uiState.update { state ->
00862         state.copy(
00863             stage = when {
00864                 state.plans.isNotEmpty() -> RemodelStage.PlansReady
00865                 state.draftAnalysis != null -> RemodelStage.AnalysisReady
00866                 state.selectedImage != null -> RemodelStage.ImageSelected
00867                 else -> RemodelStage.Idle
00868             },
00869             error = null
00870         )
00871     }
00872 }
00873
00874 fun updateGarmentType(value: String) = updateDraftAnalysis { it.copy(garmentType = value) }
00875
00876 fun updateColor(value: String) = updateDraftAnalysis { it.copy(color = value) }
00877
00878 fun updateMaterial(value: String) = updateDraftAnalysis { it.copy(material = value) }
00879
00880 fun updateStyle(value: String) = updateDraftAnalysis { it.copy(style = value) }
00881
00882 fun updateDefects(rawValue: String) {
00883     val defects = rawValue.split(",", ", ")
00884     .map { it.trim() }
00885     .filter { it.isNotEmpty() }
00886     .map { GarmentDefect(name = it) }
00887     updateDraftAnalysis { it.copy(defects = defects) }
00888 }
00889
00890 fun selectIntent(intent: RemodelIntent) {
00891     _uiState.update { state ->
00892         state.copy(
00893             selectedIntent = intent,
00894             stage = if (state.plans.isNotEmpty()) RemodelStage.EditingAnalysis else
state.stage,
00895             plans = if (state.plans.isNotEmpty()) emptyList() else state.plans,
00896             selectedPlanId = null,
00897             editingPlanId = null,
00898             previewEditOptions = PreviewEditOptions(),
00899             previewJob = null,
00900             isPreviewLoading = false,

```

```

00901         previewErrorMessage = null
00902     )
00903 }
00904 }
00905
00906 fun updateUserPreferences(value: String) {
00907     _uiState.update { state ->
00908         state.copy(
00909             userPreferences = value,
00910             stage = if (state.plans.isNotEmpty()) RemodelStage.EditingAnalysis else
state.stage,
00911             selectedPlanId = null,
00912             editingPlanId = null,
00913             previewEditOptions = PreviewEditOptions(),
00914             previewJob = null,
00915             isPreviewLoading = false,
00916             previewErrorMessage = null,
00917             plans = if (state.plans.isNotEmpty()) emptyList() else state.plans
00918         )
00919     }
00920 }
00921
00922 fun generatePlans() {
00923     val analysis = _uiState.value.draftAnalysis ?: return
00924     val intent = _uiState.value.selectedIntent ?: return
00925     val sourceImage = _uiState.value.selectedImage ?: return
00926
00927     viewModelScope.launch {
00928         val activeRepository = activeRepository()
00929         _uiState.update {
00930             it.copy(
00931                 stage = RemodelStage.GeneratingPlans,
00932                 plans = emptyList(),
00933                 error = null
00934             )
00935         }
00936
00937         try {
00938             val plans = activeRepository.generatePlans(
00939                 intent = intent,
00940                 confirmedAnalysis = analysis,
00941                 userPreferences = _uiState.value.userPreferences,
00942                 responseLanguage = _uiState.value.appLanguage
00943             )
00944
00945             if (plans.isEmpty()) {
00946                 throw ModelResponseException(
00947                     tr(
00948                         _uiState.value.appLanguage,
00949                         "The server did not return any usable remodel plans.",
00950                         "服务端未返回可用改制方案。"

```

```

00951         )
00952     )
00953 }
00954
00955     historyRepository.savePlanGeneration(
00956         sourceImage = sourceImage,
00957         analysis = analysis,
00958         intent = intent,
00959         userPreferences = _uiState.value.userPreferences,
00960         plans = plans
00961     )
00962     refreshHistory()
00963     syncHistoryInBackground()
00964
00965     _uiState.update {
00966         it.copy(
00967             stage = RemodelStage.PlansReady,
00968             analysis = analysis,
00969             draftAnalysis = analysis,
00970             plans = plans,
00971             selectedPlanId = null,
00972             editingPlanId = null,
00973             previewEditOptions = PreviewEditOptions(),
00974             previewJob = null,
00975             isPreviewLoading = false,
00976             previewErrorMessage = null,
00977             error = null
00978         )
00979     }
00980     } catch (exception: IOException) {
00981         setError(
00982             stage = RemodelStage.NetworkError,
00983             error = RemodelError(RemodelErrorType.NETWORK_ERROR,
exception.message.orEmpty())
00984         )
00985     } catch (exception: ModelResponseException) {
00986         setError(
00987             stage = RemodelStage.ModelError,
00988             error = RemodelError(RemodelErrorType.MODEL_ERROR,
exception.message.orEmpty())
00989         )
00990     }
00991 }
00992 }
00993
00994 fun openPreviewEditor(planId: String) {
00995     val selectedPlan = _uiState.value.plans.firstOrNull { it.planId == planId } ?: return
00996     _uiState.update {
00997         it.copy(
00998             selectedPlanId = selectedPlan.planId,
00999             editingPlanId = selectedPlan.planId,
01000             previewEditOptions = if (it.editingPlanId == selectedPlan.planId) {

```

```

01001         it.previewEditOptions
01002     } else {
01003         PreviewEditOptions()
01004     },
01005     previewErrorMessage = null,
01006     error = null
01007 )
01008 }
01009 }
01010
01011 fun closePreviewEditor() {
01012     _uiState.update {
01013         it.copy(
01014             editingPlanId = null
01015         )
01016     }
01017 }
01018
01019 fun updatePreviewEditSilhouette(value: PreviewEditSilhouette) {
01020     _uiState.update { it.copy(previewEditOptions = it.previewEditOptions.copy(silhouette
= value)) }
01021 }
01022
01023 fun updatePreviewEditLength(value: PreviewEditLength) {
01024     _uiState.update { it.copy(previewEditOptions = it.previewEditOptions.copy(length =
value)) }
01025 }
01026
01027 fun updatePreviewEditNeckline(value: PreviewEditNeckline) {
01028     _uiState.update { it.copy(previewEditOptions = it.previewEditOptions.copy(neckline =
value)) }
01029 }
01030
01031 fun updatePreviewEditSleeve(value: PreviewEditSleeve) {
01032     _uiState.update { it.copy(previewEditOptions = it.previewEditOptions.copy(sleeve =
value)) }
01033 }
01034
01035 fun updatePreviewEditFidelity(value: PreviewEditFidelity) {
01036     _uiState.update { it.copy(previewEditOptions = it.previewEditOptions.copy(fidelity =
value)) }
01037 }
01038
01039 fun updatePreviewEditInstructions(value: String) {
01040     _uiState.update { it.copy(previewEditOptions =
it.previewEditOptions.copy(extraInstructions = value)) }
01041 }
01042
01043 fun confirmPlan(planId: String) {
01044     val analysis = _uiState.value.draftAnalysis ?: return
01045     val selectedPlan = _uiState.value.plans.firstOrNull { it.planId == planId } ?: return
01046     val editOptions = if (_uiState.value.editingPlanId == selectedPlan.planId) {
01047         _uiState.value.previewEditOptions
01048     } else {
01049         PreviewEditOptions()
01050     }

```

```

01051
01052     viewModelScope.launch {
01053         _uiState.update {
01054             it.copy(
01055                 selectedPlanId = selectedPlan.planId,
01056                 previewJob = null,
01057                 isPreviewLoading = true,
01058                 previewErrorMessage = null,
01059                 error = null
01060             )
01061         }
01062         startPreviewGeneration(
01063             analysis = analysis,
01064             planId = selectedPlan.planId,
01065             selectedPlanTitle = selectedPlan.title,
01066             editOptions = editOptions
01067         )
01068     }
01069 }
01070
01071 fun generatePreviewFromEditor() {
01072     val editingPlanId = _uiState.value.editingPlanId ?: return
01073     confirmPlan(editingPlanId)
01074 }
01075
01076 fun resumePreviewPolling(planId: String) {
01077     val state = _uiState.value
01078     val previewJob = state.previewJob ?: return
01079     if (state.isPreviewLoading) {
01080         return
01081     }
01082     val preview = state.previewFor(planId)
01083     val isRequestedPlan = state.selectedPlanId == planId
01084     val hasActiveJobForPlan = isRequestedPlan && previewJob.status in
ActivePreviewStatuses
01085     val hasActiveRenderForPlan = preview?.renderStatus in ActivePreviewRenderStatuses
01086     if (previewJob.status !in ActivePreviewStatuses &&
01087         !hasActiveRenderForPlan &&
01088         !hasActiveJobForPlan
01089     ) {
01090         return
01091     }
01092
01093     viewModelScope.launch {
01094         _uiState.update {
01095             it.copy(
01096                 selectedPlanId = planId,
01097                 isPreviewLoading = true,
01098                 previewErrorMessage = null,
01099                 error = null
01100             )

```

```

01101     }
01102     pollPreviewJob(previewJobId = previewJob.previewJobId)
01103 }
01104 }
01105
01106 fun publishPreviewResult(planId: String) {
01107     val state = _uiState.value
01108     val selectedImage = state.selectedImage ?: return
01109     val analysis = state.analysis ?: state.draftAnalysis ?: return
01110     val selectedIntent = state.selectedIntent ?: return
01111     val selectedPlan = state.plans.firstOrNull { it.planId == planId } ?: return
01112     val preview = state.previewFor(planId) ?: return
01113
01114     if (preview.renderStatus != PreviewRenderStatus.COMPLETED) {
01115         return
01116     }
01117
01118     viewModelScope.launch {
01119         historyRepository.savePublishedRemodel(
01120             sourceImage = selectedImage,
01121             analysis = analysis,
01122             intent = selectedIntent,
01123             selectedPlan = selectedPlan,
01124             previewResult = preview
01125         )
01126         refreshHistory()
01127         syncHistoryInBackground()
01128     }
01129 }
01130
01131 fun toggleInspirationLike(itemId: String, baseLikeCount: Int) {
01132     viewModelScope.launch {
01133         val current = _uiState.value.inspirationEngagementFor(itemId)
01134         val nextLiked = !(current?.liked ?: false)
01135         val nextLikeCount = if (current == null) {
01136             if (nextLiked) baseLikeCount + 1 else baseLikeCount
01137         } else if (nextLiked) {
01138             current.likeCount + 1
01139         } else {
01140             (current.likeCount - 1).coerceAtLeast(0)
01141         }
01142         saveInspirationEngagement(
01143             InspirationEngagementRecord(
01144                 itemId = itemId,
01145                 liked = nextLiked,
01146                 bookmarked = current?.bookmarked ?: false,
01147                 likeCount = nextLikeCount,
01148                 comments = current?.comments ?: emptyList()
01149             )
01150         )
01151     }

```

```

01151     }
01152 }
01153
01154 fun toggleInspirationBookmark(itemId: String, baseLikeCount: Int) {
01155     viewModelScope.launch {
01156         val current = _uiState.value.inspirationEngagementFor(itemId)
01157         saveInspirationEngagement(
01158             InspirationEngagementRecord(
01159                 itemId = itemId,
01160                 liked = current?.liked ?: false,
01161                 bookmarked = !(current?.bookmarked ?: false),
01162                 likeCount = current?.likeCount ?: baseLikeCount,
01163                 comments = current?.comments ?: emptyList()
01164             )
01165         )
01166     }
01167 }
01168
01169 fun addInspirationComment(itemId: String, baseLikeCount: Int, message: String) {
01170     val trimmed = message.trim()
01171     if (trimmed.isBlank()) {
01172         return
01173     }
01174
01175     viewModelScope.launch {
01176         val current = _uiState.value.inspirationEngagementFor(itemId)
01177         saveInspirationEngagement(
01178             InspirationEngagementRecord(
01179                 itemId = itemId,
01180                 liked = current?.liked ?: false,
01181                 bookmarked = current?.bookmarked ?: false,
01182                 likeCount = current?.likeCount ?: baseLikeCount,
01183                 comments = listOf(
01184                     InspirationComment(
01185                         commentId = "comment-${UUID.randomUUID()}",
01186                         authorName = tr(_uiState.value.appLanguage, "You", "你"),
01187                         message = trimmed,
01188                         createdAtEpochMillis = System.currentTimeMillis()
01189                     )
01190                 ) + (current?.comments ?: emptyList())
01191             )
01192         )
01193     }
01194 }
01195
01196 fun restoreSavedPlanGeneration(recordId: String): Boolean {
01197     val savedRecord = _uiState.value.recentPlanGenerationRecords
01198         .firstOrNull { it.recordId == recordId }
01199     ?: return false
01200 }

```



```

01201     _uiState.update {
01202         it.copy(
01203             stage = RemodelStage.PlansReady,
01204             selectedImage = savedRecord.sourceImage,
01205             analysis = savedRecord.analysis,
01206             draftAnalysis = savedRecord.analysis,
01207             selectedIntent = savedRecord.intent,
01208             userPreferences = savedRecord.userPreferences,
01209             plans = savedRecord.plans,
01210             selectedPlanId = null,
01211             editingPlanId = null,
01212             previewEditOptions = PreviewEditOptions(),
01213             previewJob = null,
01214             isPreviewLoading = false,
01215             previewErrorMessage = null,
01216             error = null
01217         )
01218     }
01219     return true
01220 }
01221
01222 private fun updateDraftAnalysis(transform: (GarmentAnalysis) -> GarmentAnalysis) {
01223     _uiState.update { state ->
01224         val current = state.draftAnalysis ?: return@update state
01225         val updated = transform(current)
01226         state.copy(
01227             draftAnalysis = updated,
01228             plans = emptyList(),
01229             selectedPlanId = null,
01230             editingPlanId = null,
01231             previewEditOptions = PreviewEditOptions(),
01232             previewJob = null,
01233             isPreviewLoading = false,
01234             previewErrorMessage = null,
01235             stage = RemodelStage.EditingAnalysis
01236         )
01237     }
01238 }
01239
01240 private suspend fun startPreviewGeneration(
01241     analysis: GarmentAnalysis,
01242     planId: String,
01243     selectedPlanTitle: String,
01244     editOptions: PreviewEditOptions
01245 ) {
01246     if (planId.isBlank()) {
01247         _uiState.update {
01248             it.copy(
01249                 isPreviewLoading = false,
01250                 previewErrorMessage = tr(

```

```

01251         it.appLanguage,
01252         "The plan exists, but the current result is missing a preview
01253         identifier.",
01254         "方案已生成, 但当前结果缺少可视化预览标识。"
01255     )
01256 }
01257 return
01258 }
01259
01260 try {
01261     val createResult = activeRepository().createPreviewJob(
01262         analysisId = analysis.analysisId,
01263         planId = planId,
01264         editOptions = editOptions
01265     )
01266     _uiState.update {
01267         it.copy(
01268             previewJob = createPendingPreviewJobSnapshot(
01269                 previewJobId = createResult.previewJobId,
01270                 analysisId = analysis.analysisId,
01271                 planId = planId,
01272                 requestedPlanCount = createResult.requestedPlanCount,
01273                 pollPath = createResult.pollPath,
01274                 status = PreviewJobStatus.fromWire(createResult.status)
01275             ),
01276             selectedPlanId = planId,
01277             isPreviewLoading = true,
01278             previewErrorMessage = null,
01279             error = null
01280         )
01281     }
01282     pollPreviewJob(previewJobId = createResult.previewJobId)
01283 } catch (exception: IOException) {
01284     _uiState.update {
01285         it.copy(
01286             isPreviewLoading = false,
01287             previewErrorMessage = exception.message.orEmpty().ifBlank {
01288                 tr(it.appLanguage, "The plan is confirmed, but the final image
01289                 request failed.", "方案已确认, 但最终效果图请求失败。")
01290             }
01291         )
01292     }
01293 } catch (exception: InvalidImageException) {
01294     _uiState.update {
01295         it.copy(
01296             isPreviewLoading = false,
01297             previewErrorMessage = exception.message.orEmpty().ifBlank {
01298                 tr(it.appLanguage, "The server rejected the final image request.
01299                 Adjust the tuning options and try again.", "最终效果图请求被服务端拒绝, 请调整微调参数后重试。")
01300             }
01301         )
01302     }

```

```

01301     } catch (exception: ModelResponseException) {
01302         if (isMissingPreviewContextError(exception.message.orEmpty()) &&
01303             recoverPreviewContextAfterBackendReset(selectedPlanTitle)
01304         ) {
01305             return
01306         }
01307         _uiState.update {
01308             it.copy(
01309                 isPreviewLoading = false,
01310                 previewErrorMessage = exception.message.orEmpty().ifBlank {
01311                     tr(it.appLanguage, "The plan is confirmed, but the final image
response was invalid.", "方案已确认, 但最终效果图返回异常。")
01312                 }
01313             )
01314         }
01315     }
01316 }
01317
01318 private suspend fun pollPreviewJob(previewJobId: String) {
01319     var reachedTerminalState = false
01320     try {
01321         repeat(PreviewMaxPollAttempts) { attempt ->
01322             if (attempt > 0) {
01323                 delay(PreviewPollDelayMillis)
01324             }
01325             val previewJob = activeRepository().getPreviewJob(previewJobId)
01326             val isTerminal = previewJob.status !in ActivePreviewStatuses
01327             val filteredMessage = previewJob.results
01328                 .firstOrNull { it.renderStatus == PreviewRenderStatus.FILTERED }
01329                 ?.errorMessage
01330             _uiState.update {
01331                 it.copy(
01332                     previewJob = previewJob,
01333                     isPreviewLoading = !isTerminal,
01334                     previewErrorMessage = when {
01335                         previewJob.status == PreviewJobStatus.FAILED && filteredMessage !
= null ->
01336                             filteredMessage
01337                         previewJob.status == PreviewJobStatus.FAILED ->
01338                             tr(it.appLanguage, "The plan is confirmed, but final image
generation failed. Please try again shortly.", "方案已确认, 但最终效果图生成失败, 请稍后重试。")
01339                         previewJob.status == PreviewJobStatus.EXPIRED ->
01340                             tr(it.appLanguage, "The image job expired. Please generate it
again.", "效果图任务已过期, 请重新生成。")
01341                         previewJob.status == PreviewJobStatus.COMPLETED_WITH_FAILURES ->
01342                             filteredMessage ?: tr(it.appLanguage, "The final image did
not pass validation, so it will not be shown this time.", "最终效果图未通过校验, 本次不展示结果。")
01343                         else -> null
01344                     }
01345                 )
01346             }
01347             if (isTerminal) {
01348                 reachedTerminalState = true
01349                 return
01350             }

```

```

01351     }
01352     } catch (exception: IOException) {
01353         _uiState.update { state ->
01354             state.copy(
01355                 isPreviewLoading = false,
01356                 previewErrorMessage = if (state.previewJob?.status in
ActivePreviewStatuses) {
01357                     buildPreviewStillProcessingMessage(state.appLanguage)
01358                 } else {
01359                     exception.message.orEmpty().ifBlank {
01360                         tr(state.appLanguage, "The final image status could not be
refreshed. Please try again.", "当前无法刷新最终效果图状态, 请稍后重试。")
01361                     }
01362                 }
01363             )
01364         }
01365         return
01366     } catch (exception: ModelResponseException) {
01367         _uiState.update { state ->
01368             state.copy(
01369                 isPreviewLoading = false,
01370                 previewErrorMessage = if (state.previewJob?.status in
ActivePreviewStatuses) {
01371                     buildPreviewStillProcessingMessage(state.appLanguage)
01372                 } else {
01373                     exception.message.orEmpty().ifBlank {
01374                         tr(state.appLanguage, "The server returned an invalid final image
status. Please try again.", "服务端返回了异常的最终效果图状态, 请稍后重试。")
01375                     }
01376                 }
01377             )
01378         }
01379         return
01380     }
01381     if (!reachedTerminalState) {
01382         _uiState.update {
01383             it.copy(
01384                 isPreviewLoading = false,
01385                 previewErrorMessage = tr(
01386                     it.appLanguage,
01387                     "The plan is confirmed and the final image is still processing in the
background.",
01388                     "方案已确认, 最终效果图仍在后台处理中。"
01389                 )
01390             )
01391         }
01392     }
01393 }
01394
01395 private suspend fun recoverPreviewContextAfterBackendReset(selectedPlanTitle: String):
Boolean {
01396     val state = _uiState.value
01397     val selectedImage = state.selectedImage ?: return false
01398     val selectedIntent = state.selectedIntent ?: return false
01399     val currentPreferences = state.userPreferences
01400

```

```

01401     return try {
01402         val language = _uiState.value.appLanguage
01403         val activeRepository = activeRepository(state)
01404         val refreshedAnalysis = activeRepository.analyze(selectedImage, language)
01405         val refreshedPlans = activeRepository.generatePlans(
01406             intent = selectedIntent,
01407             confirmedAnalysis = refreshedAnalysis,
01408             userPreferences = currentPreferences,
01409             responseLanguage = language
01410         )
01411         _uiState.update {
01412             it.copy(
01413                 stage = RemodelStage.PlansReady,
01414                 analysis = refreshedAnalysis,
01415                 draftAnalysis = refreshedAnalysis,
01416                 plans = refreshedPlans,
01417                 selectedPlanId = null,
01418                 editingPlanId = null,
01419                 previewEditOptions = PreviewEditOptions(),
01420                 previewJob = null,
01421                 isPreviewLoading = false,
01422                 previewErrorMessage = buildPreviewContextResetMessage(
01423                     previousPlanTitle = selectedPlanTitle,
01424                     refreshedPlans = refreshedPlans
01425                 ),
01426                 error = null
01427             )
01428         }
01429         true
01430     } catch (_: Exception) {
01431         _uiState.update {
01432             it.copy(
01433                 isPreviewLoading = false,
01434                 selectedPlanId = null,
01435                 previewJob = null,
01436                 previewErrorMessage = tr(
01437                     it.appLanguage,
01438                     "The backend session was reset. Re-run analysis for the current image
before generating the final image again.",
01439                     "后端会话已重置，请重新识别当前图片后再生成最终效果图。"
01440                 )
01441             )
01442         }
01443         true
01444     }
01445 }
01446
01447 private fun isMissingPreviewContextError(message: String): Boolean {
01448     val normalized = message.lowercase()
01449     return normalized.contains("analysisid 不存在") ||
01450         normalized.contains("planid 不存在")

```

```

01451     }
01452
01453     private fun buildPreviewContextResetMessage(
01454         previousPlanTitle: String,
01455         refreshedPlans: List<RemodelPlan>
01456     ): String {
01457         val language = _uiState.value.appLanguage
01458         val matchedPlan = refreshedPlans.firstOrNull { it.title == previousPlanTitle }
01459         return if (matchedPlan != null) {
01460             tr(
01461                 language,
01462                 "The backend session was reset and the current image plus plans were synced
again. Confirm \"${matchedPlan.title}\" or choose another plan before generating the final image.",
01463                 "后端会话已重置，已重新同步当前图片与方案。请重新确认“${matchedPlan.title}”或选择其他方
案后，再生成最终效果图。"
01464             )
01465         } else {
01466             tr(
01467                 language,
01468                 "The backend session was reset and the current image plus plans were synced
again. Confirm a plan again before generating the final image.",
01469                 "后端会话已重置，已重新同步当前图片与方案。请重新确认一个方案后，再生成最终效果图。"
01470             )
01471         }
01472     }
01473
01474     private fun createPendingPreviewJobSnapshot(
01475         previewJobId: String,
01476         analysisId: String,
01477         planId: String,
01478         requestedPlanCount: Int,
01479         pollPath: String,
01480         status: PreviewJobStatus
01481     ) = PreviewJobSnapshot(
01482         previewJobId = previewJobId,
01483         analysisId = analysisId,
01484         status = status,
01485         requestedPlanCount = requestedPlanCount,
01486         completedPlanCount = 0,
01487         failedPlanCount = 0,
01488         results = listOf(
01489             PlanPreviewResult(
01490                 planId = planId,
01491                 renderStatus = PreviewRenderStatus.QUEUED
01492             )
01493         ),
01494         pollPath = pollPath
01495     )
01496
01497     private fun buildPreviewStillProcessingMessage(language: AppLanguage): String =
01498         tr(
01499             language,
01500             "The final image is still processing in the background.",

```

```

10189         try:
10190             result = callback(attempts)
10191             stage_results.append(
10192                 PreviewStageResult(
10193                     stage_name=stage_name,
10194                     status=PreviewStageStatus.SUCCEEDED,
10195                     attempts=attempts,
10196                     warnings=[],
10197                 )
10198             )
10199             return result
10200         except PreviewStageExecutionError as error:
10201             last_error = error
10202             if not error.retryable or attempts >= self._max_attempts:
10203                 stage_results.append(
10204                     PreviewStageResult(
10205                         stage_name=stage_name,
10206                         status=PreviewStageStatus.FAILED,
10207                         attempts=attempts,
10208                         warnings=[],
10209                         error_message=error.message,
10210                     )
10211                 )
10212                 raise
10213             raise last_error or PreviewStageExecutionError(stage_name=stage_name.value,
10214 message="stage failed")
10215     def _run_optional_stage(
10216         self,
10217         stage_name: PreviewStageName,
10218         stage_results: list[PreviewStageResult],
10219         callback: Callable[[int], T],
10220         *,
10221         fallback: T,
10222     ) -> tuple[T, bool]:
10223         attempts = 0
10224         while attempts < self._max_attempts:
10225             attempts += 1
10226             try:
10227                 result = callback(attempts)
10228                 stage_results.append(
10229                     PreviewStageResult(
10230                         stage_name=stage_name,
10231                         status=PreviewStageStatus.SUCCEEDED,
10232                         attempts=attempts,
10233                         warnings=[],
10234                     )
10235                 )
10236             return result, False
10237         except PreviewStageExecutionError as error:
10238             if error.retryable and attempts < self._max_attempts:

```

```

10239         continue
10240     stage_results.append(
10241         PreviewStageResult(
10242             stage_name=stage_name,
10243             status=PreviewStageStatus.FAILED,
10244             attempts=attempts,
10245             warnings=[],
10246             error_message=error.message,
10247         )
10248     )
10249     if isinstance(fallback, PreviewSegmentationResult):
10250         return replace(
10251             fallback,
10252             warnings=[*fallback.warnings, f"{stage_name.value}:"
10253             {error.message}],
10254             ), True
10255     return fallback, True
10256 // File: app/providers/base.py
10257 from __future__ import annotations
10258
10259 from collections.abc import Mapping
10260 from typing import Any, Protocol
10261
10262 from app.domain.models import (
10263     AnalyzeGarmentInput,
10264     GenerateRemodelPlansInput,
10265     PreviewComparisonResult,
10266     PreviewGeneratedImage,
10267     PreviewJobInput,
10268     PreviewSegmentationResult,
10269     PreviewSourceImage,
10270 )
10271
10272
10273 class RemodelProvider(Protocol):
10274     def analyze_garment(self, request: AnalyzeGarmentInput) -> Mapping[str, Any]:
10275         """Return the raw provider payload for garment analysis."""
10276
10277     def generate_remodel_plans(self, request: GenerateRemodelPlansInput) -> Mapping[str,
10278 Any]:
10279         """Return the raw provider payload for remodel plan generation."""
10280
10281 class ProviderTransport(Protocol):
10282     def post_json(self, url: str, payload: Mapping[str, Any], *, timeout_seconds: float) ->
10283 Mapping[str, Any]:
10284         """POST JSON and return a decoded mapping."""
10285
10286 class SourceImageProvider(Protocol):
10287     def read_source_image(self, request: PreviewJobInput, *, attempt: int = 1) ->
10288 PreviewSourceImage:
10289         """Return the source image asset for the preview pipeline."""

```



```

10289
10290
10291 class SegmentationProvider(Protocol):
10292     def preprocess_and_segment(
10293         self,
10294         source: PreviewSourceImage,
10295         *,
10296         plan_id: str,
10297         attempt: int = 1,
10298     ) -> PreviewSegmentationResult:
10299         """Return preprocessed source bytes and an optional segmentation mask."""
10300
10301
10302 class ImageEditProvider(Protocol):
10303     def generate_after_image(
10304         self,
10305         *,
10306         plan_id: str,
10307         prompt: str,
10308         source: PreviewSourceImage,
10309         segmentation: PreviewSegmentationResult,
10310         attempt: int = 1,
10311     ) -> PreviewGeneratedImage:
10312         """Return the generated after image for a preview job."""
10313
10314
10315 class VisualQaProvider(Protocol):
10316     def review_and_compose(
10317         self,
10318         *,
10319         plan_id: str,
10320         source: PreviewSourceImage,
10321         generated: PreviewGeneratedImage,
10322         attempt: int = 1,
10323     ) -> PreviewComparisonResult:
10324         """Return the composed comparison image plus QA metadata."""
10325
10326 // File: app/providers/qwen_chat.py
10327 from __future__ import annotations
10328
10329 from collections.abc import Mapping
10330 from dataclasses import dataclass
10331 from json import dumps
10332 from typing import Any
10333
10334 from app.domain.errors import ProviderConfigurationError, ProviderOutputValidationError
10335 from app.domain.models import AnalyzeGarmentInput, GenerateRemodelPlansInput
10336 from app.providers.base import RemodelProvider
10337 from app.providers.qwen_common import extract_chat_content, parse_json_string, post_json
10338

```

```

10339 @dataclass(frozen=True)
10340 class QwenChatConfig:
10341     api_key: str
10342     vision_base_url: str = "https://dashscope.aliyuncs.com/compatible-mode/v1"
10343     vision_model_name: str = "qwen-vl-max"
10344     plan_base_url: str = "https://dashscope.aliyuncs.com/compatible-mode/v1"
10345     plan_model_name: str = "qwen-plus"
10346     timeout_seconds: float = 60.0
10347
10348     def __post_init__(self) -> None:
10349         if not self.api_key.strip():
10350             raise ProviderConfigurationError("DashScope api key must not be blank")
10351         if not self.vision_base_url.strip():
10352             raise ProviderConfigurationError("Qwen vision base URL must not be blank")
10353         if not self.plan_base_url.strip():
10354             raise ProviderConfigurationError("Qwen plan base URL must not be blank")
10355
10356
10357 class QwenChatRemodelProvider(RemodelProvider):
10358     def __init__(self, config: QwenChatConfig) -> None:
10359         self._config = config
10360
10361     @staticmethod
10362     def _use_chinese(response_language: str | None) -> bool:
10363         return (response_language or "").strip().lower() == "zh"
10364
10365     def analyze_garment(self, request: AnalyzeGarmentInput) -> Mapping[str, Any]:
10366         if not request.source_image_url:
10367             raise ProviderConfigurationError(
10368                 "Qwen vision provider requires a source_image_url from object storage"
10369             )
10370
10371         use_chinese = self._use_chinese(request.response_language)
10372         payload = {
10373             "model": self._config.vision_model_name,
10374             "response_format": {"type": "json_object"},
10375             "temperature": 0.1,
10376             "messages": [
10377                 {
10378                     "role": "system",
10379                     "content": (
10380                         "你是 Second Bloom 的旧衣识别助手。请只输出一个 JSON 对象，不要输出
10381                         markdown。"
10382                         if use_chinese
10383                         else "You are Second Bloom's garment recognition assistant. Return
10384                         one JSON object only and no markdown."
10385                     ),
10386                 },
10387                 {
10388                     "role": "user",
10389                     "content": [

```

```

10389         "type": "image_url",
10390         "image_url": {"url": request.source_image_url},
10391     },
10392     {
10393         "type": "text",
10394         "text": (
10395             (
10396                 "请识别这件旧衣, 并严格输出 JSON, 结构为: "
10397                 '{"analysis":'
10398                 '{"analysisId": "analysis-...", "garmentType": "...", "color": "...", "material": "...", '
10399                 '"style": "...", "defects": [{"name": "...", "severity": "low|
medium|high"}], '
10400                 '"backgroundComplexity": "low|
high", "confidence": 0.0, "warnings": [{"code": "...", "message": "..."}]}'
10401                 "要求: 1. confidence 必须在 0 到 1 之间。"
10402                 "2. backgroundComplexity 只能是 low 或 high。"
10403                 "3. defects 和 warnings 没有时返回空数组。"
10404                 "4. analysisId 需使用 analysis- 前缀。"
10405                 "5. 所有识别字段与 warning message 都用简体中文输出。"
10406                 "6. 仅根据图片识别衣物, 不要虚构背景故事。"
10407             )
10408             if use_chinese
10409             else (
10410                 "Analyze this garment and return strict JSON with the
structure "
10411                 '{"analysis":'
10412                 '{"analysisId": "analysis-...", "garmentType": "...", "color": "...", "material": "...", '
10413                 '"style": "...", "defects": [{"name": "...", "severity": "low|
medium|high"}], '
10414                 '"backgroundComplexity": "low|
high", "confidence": 0.0, "warnings": [{"code": "...", "message": "..."}]}'
10415                 "Requirements: 1. confidence must be between 0 and 1. "
10416                 "2. backgroundComplexity must be low or high only. "
10417                 "3. defects and warnings must be empty arrays when
absent. "
10418                 "4. analysisId must use the analysis- prefix. "
10419                 "5. All garment fields and warning messages must be
written in English. "
10420                 "6. Base the answer only on the image and do not invent a
story."
10421             )
10422         ),
10423     },
10424 ],
10425 "max_tokens": 800,
10426 }
10427 response = post_json(
10428     url=f"{self._config.vision_base_url.rstrip('/')}/chat/completions",
10429     payload=payload,
10430     api_key=self._config.api_key,
10431     timeout_seconds=self._config.timeout_seconds,
10432 )
10433 content = extract_chat_content(response)
10434 parsed = parse_json_string(content)
10435 analysis = parsed.get("analysis")
10436 if not isinstance(analysis, Mapping):
10437     raise ProviderOutputValidationError("Qwen vision response missing analysis
object")
10438 return {"analysis": dict(analysis)}

```

```

10439
10440 def generate_remodel_plans(self, request: GenerateRemodelPlansInput) -> Mapping[str,
Any]:
10441     analysis = request.confirmed_analysis
10442     use_chinese = self._use_chinese(request.response_language)
10443     defects = [
10444         {"name": item.name, "severity": item.severity}
10445         for item in analysis.defects
10446     ]
10447     warnings = [
10448         {"code": item.code.value, "message": item.message}
10449         for item in analysis.warnings
10450     ]
10451     payload = {
10452         "model": self._config.plan_model_name,
10453         "response_format": {"type": "json_object"},
10454         "temperature": 0.4,
10455         "messages": [
10456             {
10457                 "role": "system",
10458                 "content": (
10459                     "你是 Second Bloom 的旧衣改制顾问。请只输出一个 JSON 对象，不要输出
markdown。"
10460                     if use_chinese
10461                     else "You are Second Bloom's garment remodel consultant. Return one
JSON object only and no markdown."
10462                 ),
10463             },
10464             {
10465                 "role": "user",
10466                 "content": (
10467                     (
10468                         "请基于以下识别结果生成恰好 3 套旧衣改制方案，并严格输出 JSON。"
10469                         "输出结构为："
10470                         '{"plans":[{"title":"...", "summary":"...", "difficulty":"easy|
medium|hard", "materials":["..."], '
10471                         '{"estimatedTime":"...", "steps":
[{"title":"...", "detail":"..."}]}], '
10472                         '"reasoningNote":"..."。'
10473                         "要求：1. plans 数组必须恰好 3 项。"
10474                         "2. 每套方案都要可执行，且至少 2 个步骤、至少 2 种材料。"
10475                         "3. 不要返回 planId，服务端会生成。"
10476                         "4. 方案之间要有明显差异。"
10477                         "5. title、summary、materials、estimatedTime、steps、reasoningNote
全部使用简体中文。"
10478                         f" 识别结果：{{\{"analysisId\":"{analysis.analysis_id}\", "
10479                         f"\{"garmentType\":"{analysis.garment_type}\", "
10480                         f"\{"color\":"{analysis.color}\", "
10481                         f"\{"material\":"{analysis.material}\", "
10482                         f"\{"style\":"{analysis.style}\", "
10483                         f"\{"backgroundComplexity\":"
f"\{analysis.background_complexity.value}\", "
10484                         f"\{"defects\":{dumps(defects, ensure_ascii=False)}, "
10485                         f"\{"warnings\":{dumps(warnings, ensure_ascii=False)}}}}。"
10486                         f" 改制目标：{request.intent.value if hasattr(request.intent,
'value') else request.intent}。"
10487                         f" 用户偏好：{request.user_preferences or '无额外偏好'}。"
10488                     )
                )
            }
        ]
    }

```

```

10489         if use_chinese
10490         else (
10491             "Using the garment analysis below, generate exactly 3 remodel
plans and return strict JSON. "
10492             'The schema must be {"plans":
[{"title":"...", "summary":"...", "difficulty":"easy|medium|hard", "materials":["..."], '
10493             '"estimatedTime":"...", "steps":
[{"title":"...", "detail":"..."}]}, "reasoningNote":"..."}.'
10494             "Requirements: 1. The plans array must contain exactly 3 items. "
10495             "2. Every plan must be actionable with at least 2 steps and 2
materials. "
10496             "3. Do not return planId because the server will generate it. "
10497             "4. The three plans must be meaningfully different from each
other. "
10498             "5. title, summary, materials, estimatedTime, steps, and
reasoningNote must all be written in English. "
10499             f'Analysis:
{"analysisId":"{analysis.analysis_id}", "garmentType":"{analysis.garment_type}", "color":"{analysis.c
olor}", '
10500             f'"material":"{analysis.material}", "style":"{analysis.style}", "backgroundComplexity":"{analysis.back
ground_complexity.value}", '
10501             f'"defects":{dumps(defects, ensure_ascii=False)}, "warnings":
{dumps(warnings, ensure_ascii=False)}}.'
10502             f'Remodel intent: {request.intent.value if
hasattr(request.intent, "value") else request.intent}.'
10503             f'User preferences: {request.user_preferences or "none"}.'
10504         )
10505     ),
10506 },
10507 ],
10508     "max_tokens": 1400,
10509 }
10510 response = post_json(
10511     url=f"{self._config.plan_base_url.rstrip('/')}/chat/completions",
10512     payload=payload,
10513     api_key=self._config.api_key,
10514     timeout_seconds=self._config.timeout_seconds,
10515 )
10516 content = extract_chat_content(response)
10517 parsed = parse_json_string(content)
10518 if not isinstance(parsed.get("plans"), list):
10519     raise ProviderOutputValidationError("Qwen plan response missing plans array")
10520 return parsed
10521 // File: app/providers/qwen_image_edit.py
10522 from __future__ import annotations
10523
10524 from dataclasses import dataclass
10525 from io import BytesIO
10526
10527 from PIL import Image
10528
10529 from app.domain.errors import PreviewStageExecutionError, ProviderConfigurationError,
ProviderTransportError
10530 from app.domain.models import PreviewGeneratedImage, PreviewSegmentationResult,
PreviewSourceImage
10531 from app.providers.qwen_common import (
10532     download_bytes,
10533     extract_error_message,
10534     extract_multimodal_image_url,
10535     post_json,
10536 )
10537
10538

```

```

10539 @dataclass(frozen=True)
10540 class QwenImageEditConfig:
10541     api_key: str
10542     endpoint_url: str = "https://dashscope.aliyuncs.com/api/v1/services/aigc/multimodal-
generation/generation"
10543     model_name: str = "qwen-image-edit-max"
10544     fallback_model_name: str = "qwen-image-2.0-pro"
10545     negative_prompt: str = (
10546         "不要改变人物脸部、发型、肢体和背景场景。"
10547         "禁止额外人物、额外衣物、拼贴、字幕、水印、夸张变形、错误手指、明显 AI 痕迹。"
10548     )
10549     prompt_extend: bool = True
10550     watermark: bool = False
10551     output_count: int = 1
10552     timeout_seconds: float = 120.0
10553     target_long_edge: int = 1536
10554
10555     def __post_init__(self) -> None:
10556         if not self.api_key.strip():
10557             raise ProviderConfigurationError("Qwen image edit api key must not be blank")
10558         if not self.endpoint_url.strip():
10559             raise ProviderConfigurationError("Qwen image edit endpoint must not be blank")
10560         if not self.model_name.strip():
10561             raise ProviderConfigurationError("Qwen image edit model_name must not be blank")
10562
10563
10564 class QwenImageEditProvider:
10565     def __init__(self, config: QwenImageEditConfig) -> None:
10566         self._config = config
10567
10568     def generate_after_image(
10569         self,
10570         *,
10571         plan_id: str,
10572         prompt: str,
10573         source: PreviewSourceImage,
10574         segmentation: PreviewSegmentationResult,
10575         attempt: int = 1,
10576     ) -> PreviewGeneratedImage:
10577         if not source.source_url:
10578             raise PreviewStageExecutionError(
10579                 "generate_after_image",
10580                 "Qwen image edit requires a source image URL from object storage",
10581             )
10582
10583         models = [self._config.model_name]
10584         fallback = self._config.fallback_model_name.strip()
10585         if fallback and fallback not in models:
10586             models.append(fallback)
10587
10588         last_error: str | None = None

```

```

10589         for model_name in models:
10590             try:
10591                 image_url = self._generate_image_url(
10592                     model_name=model_name,
10593                     prompt=prompt,
10594                     source_url=source.source_url,
10595                     source_bytes=source.image_bytes,
10596                 )
10597                 image_bytes = download_bytes(image_url,
10598                     timeout_seconds=self._config.timeout_seconds)
10599                 warnings: list[str] = []
10600                 if model_name != self._config.model_name:
10601                     warnings.append(f"used fallback image edit model {model_name}")
10602                 if attempt > 1:
10603                     warnings.append("image edit succeeded after retry")
10604                 return PreviewGeneratedImage(
10605                     image_bytes=image_bytes,
10606                     prompt=prompt,
10607                     warnings=warnings,
10608                     remote_url=image_url,
10609                 )
10610             except (ProviderTransportError, PreviewStageExecutionError) as error:
10611                 last_error = str(error)
10612                 continue
10613         raise PreviewStageExecutionError(
10614             "generate_after_image",
10615             last_error or "Qwen image edit failed without details",
10616         )
10617
10618     def _generate_image_url(
10619         self,
10620         *,
10621         model_name: str,
10622         prompt: str,
10623         source_url: str,
10624         source_bytes: bytes,
10625     ) -> str:
10626         parameters = {
10627             "n": self._config.output_count,
10628             "negative_prompt": self._config.negative_prompt,
10629             "prompt_extend": self._config.prompt_extend,
10630             "watermark": self._config.watermark,
10631         }
10632         size_hint = self._resolve_size_hint(source_bytes)
10633         if size_hint is not None:
10634             parameters["size"] = size_hint
10635         payload = {
10636             "model": model_name,
10637             "input": {
10638                 "messages": [

```

```

10639         {
10640             "role": "user",
10641             "content": [
10642                 {"image": source_url},
10643                 {"text": prompt},
10644             ],
10645         }
10646     ],
10647 },
10648     "parameters": parameters,
10649 }
10650 response = post_json(
10651     url=self._config.endpoint_url,
10652     payload=payload,
10653     api_key=self._config.api_key,
10654     timeout_seconds=self._config.timeout_seconds,
10655 )
10656 try:
10657     return extract_multimodal_image_url(response)
10658 except ProviderTransportError as error:
10659     message = extract_error_message(response) or error.message
10660     raise PreviewStageExecutionError("generate_after_image", message) from error
10661
10662 def _resolve_size_hint(self, source_bytes: bytes) -> str | None:
10663     try:
10664         with Image.open(BytesIO(source_bytes)) as image:
10665             width, height = image.size
10666     except Exception:
10667         return None
10668
10669     if width <= 0 or height <= 0:
10670         return None
10671
10672     long_edge = max(width, height)
10673     scale = self._config.target_long_edge / long_edge if long_edge >
self._config.target_long_edge else 1.0
10674     scaled_width = _round_to_nearest_16(_clamp_int(width * scale))
10675     scaled_height = _round_to_nearest_16(_clamp_int(height * scale))
10676     scaled_width = min(max(scaled_width, 512), 2048)
10677     scaled_height = min(max(scaled_height, 512), 2048)
10678     return f"{scaled_width}*{scaled_height}"
10679
10680
10681 def _clamp_int(value: float) -> int:
10682     return max(1, int(round(value)))
10683
10684
10685 def _round_to_nearest_16(value: int) -> int:
10686     return max(16, int(round(value / 16.0) * 16))
10687 // File: app/providers/qwen_visual_qa.py
10688 from __future__ import annotations

```



```

10689
10690 from dataclasses import dataclass
10691 from json import dumps
10692
10693 from app.domain.errors import PreviewStageExecutionError, ProviderConfigurationError,
ProviderOutputValidationError
10694 from app.domain.models import PreviewComparisonResult, PreviewGeneratedImage,
PreviewSourceImage
10695 from app.providers.qwen_common import extract_chat_content, parse_json_string, post_json
10696
10697
10698 @dataclass(frozen=True)
10699 class QwenVisualQaConfig:
10700     api_key: str
10701     base_url: str = "https://dashscope.aliyuncs.com/compatible-mode/v1"
10702     model_name: str = "qwen-vl-max"
10703     timeout_seconds: float = 60.0
10704     minimum_score: float = 0.6
10705
10706     def __post_init__(self) -> None:
10707         if not self.api_key.strip():
10708             raise ProviderConfigurationError("Qwen visual QA api key must not be blank")
10709         if not self.base_url.strip():
10710             raise ProviderConfigurationError("Qwen visual QA base_url must not be blank")
10711         if not self.model_name.strip():
10712             raise ProviderConfigurationError("Qwen visual QA model_name must not be blank")
10713
10714
10715 class QwenVisualQaProvider:
10716     def __init__(self, config: QwenVisualQaConfig) -> None:
10717         self._config = config
10718
10719     def review_and_compose(
10720         self,
10721         *,
10722         plan_id: str,
10723         source: PreviewSourceImage,
10724         generated: PreviewGeneratedImage,
10725         attempt: int = 1,
10726     ) -> PreviewComparisonResult:
10727         if not source.source_url or not generated.remote_url:
10728             raise PreviewStageExecutionError(
10729                 "qa_compose_comparison",
10730                 "Qwen visual QA requires both source and generated image URLs",
10731             )
10732
10733         payload = {
10734             "model": self._config.model_name,
10735             "response_format": {"type": "json_object"},
10736             "temperature": 0.1,
10737             "messages": [
10738                 {

```

```

10739         "role": "system",
10740         "content": (
10741             "你是 Second Bloom 的效果图质检助手。"
10742             "请只输出 JSON, 不要输出 markdown。"
10743         ),
10744     },
10745     {
10746         "role": "user",
10747         "content": [
10748             {"type": "image_url", "image_url": {"url": source.source_url}},
10749             {"type": "image_url", "image_url": {"url": generated.remote_url}},
10750             {
10751                 "type": "text",
10752                 "text": (
10753                     "请对第二张图做保守放行式质检, 判断它是否仍然基于第一张图的同一件旧衣与同
10754                     一场景完成改制。"
10755                     "允许因为改制方案出现合理的衣服结构变化, 例如袖型、长度、领口、拼接、装饰
10756                     和版型调整。"
10757                     "请不要因为正常的改制变化就判定失败。"
10758                     "只有在以下情况才判定 pass=false: "
10759                     "1. 人物身份、脸部、发型、肢体或姿态被明显改坏; "
10760                     "2. 背景场景、镜头角度或构图被明显重绘; "
10761                     "3. 衣物类别、主色系或材质质感与原图严重脱离; "
10762                     "4. 出现额外人物、额外衣物、拼贴、字幕或水印。"
10763                     "请输出 JSON: "
10764                     '{"pass":true,"score":0.0,"reason":"..."}。'
10765                     "score 取值 0 到 1。"
10766                     "若只是轻微构图变化或正常改制导致的局部结构变化, 请保持 pass=true。"
10767                 ),
10768             },
10769         ],
10770         "max_tokens": 300,
10771     }
10772 response = post_json(
10773     url=f"{self._config.base_url.rstrip('/')}/chat/completions",
10774     payload=payload,
10775     api_key=self._config.api_key,
10776     timeout_seconds=self._config.timeout_seconds,
10777 )
10778 content = extract_chat_content(response)
10779 parsed = parse_json_string(content)
10780 passed = parsed.get("pass")
10781 score = parsed.get("score")
10782 reason = parsed.get("reason")
10783
10784 if not isinstance(passed, bool):
10785     raise ProviderOutputValidationError("visual QA response missing boolean pass
10786     field")
10787 if not isinstance(score, (int, float)):
10788     raise ProviderOutputValidationError("visual QA response missing numeric score
10789     field")
10790 score_value = float(score)

```

```

10789         reason_text = reason.strip() if isinstance(reason, str) else "视觉校验未提供原因。"
10790         if not passed or score_value < self._config.minimum_score:
10791             raise PreviewStageExecutionError("qa_compose_comparison", reason_text)
10792
10793         comparison_bytes = dumps(
10794             {"score": score_value, "reason": reason_text, "planId": plan_id},
10795             ensure_ascii=False,
10796         ).encode("utf-8")
10797         return PreviewComparisonResult(
10798             comparison_image_bytes=comparison_bytes,
10799             warnings=[],
10800             qa_score=score_value,
10801         )
10802     // File: app/db/storage.py
10803     from __future__ import annotations
10804
10805     from dataclasses import dataclass
10806     from pathlib import Path
10807     import json
10808     import re
10809     from typing import Any, Optional
10810     from urllib import error as urllib_error
10811     from urllib import parse as urllib_parse
10812     from urllib import request as urllib_request
10813
10814     import boto3
10815     from botocore.config import Config
10816
10817     def _normalize_filename(file_name: str) -> str:
10818         original_name = Path(file_name).name.strip()
10819         stem = Path(original_name).stem
10820         suffix = Path(original_name).suffix.lower()
10821
10822         safe_stem = re.sub(r"^[A-Za-z0-9._-]+", "-", stem).strip("-_.")
10823         if not safe_stem:
10824             safe_stem = "file"
10825
10826         return f"{safe_stem.lower()}_{suffix}"
10827
10828
10829     def build_object_key(prefix: str, identifier: str, file_name: str) -> str:
10830         prefix = prefix.strip("/").strip()
10831         identifier = identifier.strip("/").strip()
10832         normalized_name = _normalize_filename(file_name)
10833         return "/".join(part for part in (prefix, identifier, normalized_name) if part)
10834
10835
10836     def _build_preview_asset_key(
10837         preview_job_id: str,

```

```

10839     plan_id: str,
10840     asset_kind: str,
10841     file_name: str,
10842 ) -> str:
10843     return "/".join(
10844         part
10845         for part in (
10846             "preview-jobs",
10847             preview_job_id.strip("/").strip(),
10848             "plans",
10849             plan_id.strip("/").strip(),
10850             asset_kind.strip("/").strip(),
10851             _normalize_filename(file_name),
10852         )
10853         if part
10854     )
10855
10856
10857 def build_preview_before_asset_key(preview_job_id: str, plan_id: str, file_name: str) -> str:
10858     return _build_preview_asset_key(preview_job_id, plan_id, "before", file_name)
10859
10860
10861 def build_preview_after_asset_key(preview_job_id: str, plan_id: str, file_name: str) -> str:
10862     return _build_preview_asset_key(preview_job_id, plan_id, "after", file_name)
10863
10864
10865 def build_preview_comparison_asset_key(preview_job_id: str, plan_id: str, file_name: str) ->
10866 str:
10867     return _build_preview_asset_key(preview_job_id, plan_id, "comparison", file_name)
10868
10869
10869 @dataclass(frozen=True)
10870 class ObjectStorageConfig:
10871     endpoint_url: str
10872     access_key: str
10873     secret_key: str
10874     region_name: str = "us-east-1"
10875     force_path_style: bool = True
10876
10877
10878 @dataclass(frozen=True)
10879 class BlobObject:
10880     pathname: str
10881     url: str
10882     download_url: str
10883     content_type: str | None = None
10884     content_disposition: str | None = None
10885     etag: str | None = None
10886
10887
10888 @dataclass(frozen=True)

```

```

10889 class VercelBlobConfig:
10890     read_write_token: str
10891     api_base_url: str = "https://vercel.com/api/blob"
10892     access: str = "public"
10893
10894     def __post_init__(self) -> None:
10895         if not self.read_write_token.strip():
10896             raise ValueError("Vercel Blob read-write token must not be blank")
10897         if self.access not in {"public", "private"}:
10898             raise ValueError("Vercel Blob access must be public or private")
10899
10900     @property
10901     def store_id(self) -> str:
10902         parts = self.read_write_token.split("_")
10903         if len(parts) < 4 or not parts[3].strip():
10904             raise ValueError("Unable to extract Vercel Blob store id from token")
10905         return parts[3].strip()
10906
10907
10908 class S3ObjectStorageClient:
10909     def __init__(self, config: ObjectStorageConfig) -> None:
10910         self._config = config
10911         self._client = boto3.client(
10912             "s3",
10913             endpoint_url=config.endpoint_url,
10914             aws_access_key_id=config.access_key,
10915             aws_secret_access_key=config.secret_key,
10916             region_name=config.region_name,
10917             config=Config(
10918                 s3={"addressing_style": "path" if config.force_path_style else "virtual"}
10919             ),
10920         )
10921
10922     def ensure_bucket(self, bucket: str) -> None:
10923         existing = self._client.list_buckets()
10924         if any(entry["Name"] == bucket for entry in existing.get("Buckets", [])):
10925             return
10926
10927         self._client.create_bucket(Bucket=bucket)
10928
10929     def upload_bytes(
10930         self,
10931         *,
10932         bucket: str,
10933         key: str,
10934         body: bytes,
10935         content_type: str,
10936         metadata: Optional[dict[str, str]] = None,
10937     ) -> str:
10938         self._client.put_object(

```

```

10939         Bucket=bucket,
10940         Key=key,
10941         Body=body,
10942         ContentType=content_type,
10943         Metadata=metadata or {},
10944     )
10945     return f"s3://{bucket}/{key}"
10946
10947     def upload_json(
10948         self,
10949         *,
10950         bucket: str,
10951         key: str,
10952         payload: Any,
10953         metadata: Optional[dict[str, str]] = None,
10954     ) -> str:
10955         return self.upload_bytes(
10956             bucket=bucket,
10957             key=key,
10958             body=json.dumps(payload, ensure_ascii=False, separators=(",",
10959 ":", ")).encode("utf-8"),
10959             content_type="application/json; charset=utf-8",
10960             metadata=metadata,
10961         )
10962
10963     def download_bytes(self, *, bucket: str, key: str) -> bytes:
10964         response = self._client.get_object(Bucket=bucket, Key=key)
10965         try:
10966             return response["Body"].read()
10967         finally:
10968             response["Body"].close()
10969
10970     def generate_presigned_get_url(
10971         self,
10972         *,
10973         bucket: str,
10974         key: str,
10975         expires_in_seconds: int,
10976         response_content_type: Optional[str] = None,
10977         response_content_disposition: Optional[str] = None,
10978     ) -> str:
10979         self._validate_expires_in_seconds(expires_in_seconds)
10980         params: dict[str, Any] = {"Bucket": bucket, "Key": key}
10981         if response_content_type is not None:
10982             params["ResponseContentType"] = response_content_type
10983         if response_content_disposition is not None:
10984             params["ResponseContentDisposition"] = response_content_disposition
10985         return self._client.generate_presigned_url(
10986             "get_object",
10987             Params=params,
10988             ExpiresIn=expires_in_seconds,

```

```

10989     )
10990
10991     def generate_presigned_put_url(
10992         self,
10993         *,
10994         bucket: str,
10995         key: str,
10996         expires_in_seconds: int,
10997         content_type: Optional[str] = None,
10998         metadata: Optional[dict[str, str]] = None,
10999     ) -> str:
11000         self._validate_expires_in_seconds(expires_in_seconds)
11001         params: dict[str, Any] = {"Bucket": bucket, "Key": key}
11002         if content_type is not None:
11003             params["ContentType"] = content_type
11004         if metadata is not None:
11005             params["Metadata"] = metadata
11006         return self._client.generate_presigned_url(
11007             "put_object",
11008             Params=params,
11009             ExpiresIn=expires_in_seconds,
11010         )
11011
11012     @staticmethod
11013     def _validate_expires_in_seconds(expires_in_seconds: int) -> None:
11014         if expires_in_seconds <= 0:
11015             raise ValueError("expires_in_seconds must be positive")
11016
11017
11018     class VercelBlobClient:
11019         _API_VERSION = "9"
11020
11021         def __init__(self, config: VercelBlobConfig) -> None:
11022             self._config = config
11023
11024         def put_bytes(
11025             self,
11026             *,
11027             pathname: str,
11028             body: bytes,
11029             content_type: str,
11030             add_random_suffix: bool = False,
11031         ) -> BlobObject:
11032             query = urllib_parse.urlencode({"pathname": pathname})
11033             request = urllib_request.Request(
11034                 url=f"{self._config.api_base_url}?{query}",
11035                 data=body,
11036                 method="PUT",
11037                 headers={
11038                     "Authorization": f"Bearer {self._config.read_write_token}",

```

```

11039         "x-api-version": self._API_VERSION,
11040         "x-vercel-blob-access": self._config.access,
11041         "x-content-type": content_type,
11042         "x-add-random-suffix": "1" if add_random_suffix else "0",
11043         "x-content-length": str(len(body)),
11044     },
11045 )
11046 payload = self._request_json(request)
11047 return BlobObject(
11048     pathname=str(payload["pathname"]),
11049     url=str(payload["url"]),
11050     download_url=str(payload["downloadUrl"]),
11051     content_type=_optional_str(payload.get("contentType")),
11052     content_disposition=_optional_str(payload.get("contentDisposition")),
11053     etag=_optional_str(payload.get("etag")),
11054 )
11055
11056 def delete(self, *, url_or_pathname: str) -> None:
11057     request = urllib_request.Request(
11058         url=f"{self._config.api_base_url}/delete",
11059         data=json.dumps({"urls": [url_or_pathname]}).encode("utf-8"),
11060         method="POST",
11061         headers={
11062             "Authorization": f"Bearer {self._config.read_write_token}",
11063             "x-api-version": self._API_VERSION,
11064             "Content-Type": "application/json",
11065         },
11066     )
11067     self._request_json(request)
11068
11069 def public_url(self, pathname: str) -> str:
11070     quoted_path = urllib_parse.quote(pathname.lstrip("/"), safe="/")
11071     return (
11072         f"https://{self._config.store_id}.{self._config.access}.blob.vercel-storage.com/"
11073         f"{quoted_path}"
11074     )
11075
11076 def download_bytes(self, *, url_or_pathname: str, timeout_seconds: float = 30.0) ->
bytes:
11077     url = (
11078         url_or_pathname
11079         if url_or_pathname.startswith("http://") or
url_or_pathname.startswith("https://")
11080         else self.public_url(url_or_pathname)
11081     )
11082     request = urllib_request.Request(url=url, method="GET")
11083     try:
11084         with urllib_request.urlopen(request, timeout=timeout_seconds) as response:
11085             return response.read()
11086     except urllib_error.HTTPError as error:
11087         body = error.read().decode("utf-8", errors="replace")
11088         raise ValueError(

```



```

11089         f"Failed to download Vercel Blob object: HTTP {error.code} {body}".strip()
11090     ) from error
11091 except urllib_error.URLError as error:
11092     raise ValueError("Failed to download Vercel Blob object") from error
11093
11094 def _request_json(self, request: urllib_request.Request) -> dict[str, Any]:
11095     try:
11096         with urllib_request.urlopen(request, timeout=60.0) as response:
11097             payload = json.loads(response.read().decode("utf-8"))
11098     except urllib_error.HTTPError as error:
11099         body = error.read().decode("utf-8", errors="replace")
11100         raise ValueError(
11101             f"Vercel Blob request failed with HTTP {error.code}: {body}".strip()
11102         ) from error
11103     except urllib_error.URLError as error:
11104         raise ValueError("Vercel Blob request failed") from error
11105
11106     if not isinstance(payload, dict):
11107         raise ValueError("Vercel Blob returned a non-object response")
11108     return payload
11109
11110
11111 def _optional_str(value: Any) -> str | None:
11112     if isinstance(value, str) and value.strip():
11113         return value.strip()
11114     return None
11115
11116 // File: app/db/repository.py
11117 from __future__ import annotations
11118
11119 from collections.abc import Sequence
11120
11121 from sqlalchemy import select
11122 from sqlalchemy.orm import Session
11123
11124 from app.db.models.analysis import AnalysisRequestRecord, AnalysisResultRecord
11125 from app.db.models.plan import PlanRequestRecord, PlanResultRecord
11126 from app.db.models.preview import PreviewAssetRecord, PreviewJobRecord, PreviewResultRecord
11127
11128 class OperationalRepository:
11129     def __init__(self, session: Session) -> None:
11130         self._session = session
11131
11132     def add_analysis_request(self, record: AnalysisRequestRecord) -> AnalysisRequestRecord:
11133         self._session.add(record)
11134         self._session.flush()
11135         return record
11136
11137     def add_analysis_result(self, record: AnalysisResultRecord) -> AnalysisResultRecord:
11138         self._session.add(record)

```

```

11139         self._session.flush()
11140         return record
11141
11142     def add_plan_request(self, record: PlanRequestRecord) -> PlanRequestRecord:
11143         self._session.add(record)
11144         self._session.flush()
11145         return record
11146
11147     def add_plan_result(self, record: PlanResultRecord) -> PlanResultRecord:
11148         self._session.add(record)
11149         self._session.flush()
11150         return record
11151
11152     def add_preview_job(self, record: PreviewJobRecord) -> PreviewJobRecord:
11153         self._session.add(record)
11154         self._session.flush()
11155         return record
11156
11157     def add_preview_result(self, record: PreviewResultRecord) -> PreviewResultRecord:
11158         self._session.add(record)
11159         self._session.flush()
11160         return record
11161
11162     def add_preview_asset(self, record: PreviewAssetRecord) -> PreviewAssetRecord:
11163         self._session.add(record)
11164         self._session.flush()
11165         return record
11166
11167     def get_analysis_bundle(
11168         self,
11169         analysis_id: str,
11170     ) -> tuple[AnalysisRequestRecord, AnalysisResultRecord] | None:
11171         row = self._session.execute(
11172             select(AnalysisRequestRecord, AnalysisResultRecord)
11173             .join(
11174                 AnalysisResultRecord,
11175                 AnalysisResultRecord.analysis_id == AnalysisRequestRecord.analysis_id,
11176             )
11177             .where(AnalysisRequestRecord.analysis_id == analysis_id)
11178             ).first()
11179         if row is None:
11180             return None
11181         return row[0], row[1]
11182
11183     def get_plan_bundle(
11184         self,
11185         plan_id: str,
11186     ) -> tuple[PlanRequestRecord, PlanResultRecord] | None:
11187         row = self._session.execute(
11188             select(PlanRequestRecord, PlanResultRecord)

```

```

11189         .join(PlanResultRecord, PlanResultRecord.plan_request_id == PlanRequestRecord.id)
11190         .where(PlanResultRecord.plan_id == plan_id)
11191     ).first()
11192     if row is None:
11193         return None
11194     return row[0], row[1]
11195
11196     def get_preview_job(self, preview_job_id: str) -> PreviewJobRecord | None:
11197         return self._session.get(PreviewJobRecord, preview_job_id)
11198
11199     def list_preview_results(self, preview_job_id: str) -> Sequence[PreviewResultRecord]:
11200         return self._session.execute(
11201             select(PreviewResultRecord)
11202             .where(PreviewResultRecord.preview_job_id == preview_job_id)
11203             .order_by(PreviewResultRecord.ordinal.asc())
11204         ).scalars().all()
11205
11206     def list_preview_assets(self, preview_result_ids: Sequence[object]) ->
Sequence[PreviewAssetRecord]:
11207         if not preview_result_ids:
11208             return []
11209         return self._session.execute(
11210             select(PreviewAssetRecord)
11211             .where(PreviewAssetRecord.preview_result_id.in_(preview_result_ids))
11212         ).scalars().all()
11213
11214     def commit(self) -> None:
11215         self._session.commit()
11216
11217     def rollback(self) -> None:
11218         self._session.rollback()
11219 // File: app/db/models/analysis.py
11220 from __future__ import annotations
11221
11222 from datetime import datetime
11223 from typing import Any, Optional
11224 from uuid import UUID, uuid4
11225
11226 from sqlalchemy import BigInteger, CheckConstraint, DateTime, ForeignKey, String, Text, Uuid,
func, text
11227 from sqlalchemy.dialects.postgresql import JSONB
11228 from sqlalchemy.orm import Mapped, mapped_column
11229
11230 from app.db.base import Base
11231
11232
11233 class AnalysisRequestRecord(Base):
11234     __tablename__ = "analysis_requests"
11235     __table_args__ = (
11236         CheckConstraint(
11237             "status in ('pending', 'succeeded', 'failed')",
11238             name="analysis_requests_status_check",

```

```

11239     ),
11240     CheckConstraint(
11241         "background_complexity is null or background_complexity in ('low', 'high')",
11242         name="analysis_requests_background_complexity_check",
11243     ),
11244 )
11245
11246 id: Mapped[UUID] = mapped_column(Uuid(as_uuid=True), primary_key=True, default=uuid4)
11247 analysis_id: Mapped[str] = mapped_column(String(128), unique=True, nullable=False)
11248 request_received_at: Mapped[datetime] = mapped_column(
11249     DateTime(timezone=True),
11250     nullable=False,
11251     server_default=func.now(),
11252 )
11253 source_filename: Mapped[str] = mapped_column(String(512), nullable=False)
11254 source_mime_type: Mapped[str] = mapped_column(String(128), nullable=False)
11255 source_size_bytes: Mapped[Optional[int]] = mapped_column(BigInteger, nullable=True)
11256 image_object_key: Mapped[str] = mapped_column(Text, nullable=False)
11257 background_complexity: Mapped[Optional[str]] = mapped_column(String(16), nullable=True)
11258 provider_name: Mapped[str] = mapped_column(String(64), nullable=False)
11259 provider_model: Mapped[str] = mapped_column(String(128), nullable=False)
11260 status: Mapped[str] = mapped_column(String(32), nullable=False)
11261 error_code: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
11262 error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
11263 latency_ms: Mapped[Optional[int]] = mapped_column(BigInteger, nullable=True)
11264
11265
11266 class AnalysisResultRecord(Base):
11267     __tablename__ = "analysis_results"
11268     __table_args__ = (
11269         CheckConstraint(
11270             "background_complexity in ('low', 'high')",
11271             name="analysis_results_background_complexity_check",
11272         ),
11273         CheckConstraint(
11274             "confidence >= 0 and confidence <= 1",
11275             name="analysis_results_confidence_check",
11276         ),
11277     )
11278
11279 analysis_id: Mapped[str] = mapped_column(
11280     String(128),
11281     ForeignKey("analysis_requests.analysis_id", ondelete="CASCADE"),
11282     primary_key=True,
11283 )
11284 garment_type: Mapped[str] = mapped_column(String(256), nullable=False)
11285 color: Mapped[str] = mapped_column(String(256), nullable=False)
11286 material: Mapped[str] = mapped_column(String(256), nullable=False)
11287 style: Mapped[str] = mapped_column(String(256), nullable=False)
11288 background_complexity: Mapped[str] = mapped_column(String(16), nullable=False)

```

```

11289 confidence: Mapped[float] = mapped_column(nullable=False)
11290 warnings_json: Mapped[list[dict[str, str]]] = mapped_column(
11291     JSONB,
11292     nullable=False,
11293     server_default=text("'[]::jsonb"),
11294 )
11295 defects_json: Mapped[list[dict[str, Optional[str]]]] = mapped_column(
11296     JSONB,
11297     nullable=False,
11298     server_default=text("'[]::jsonb"),
11299 )
11300 raw_provider_payload_json: Mapped[Optional[dict[str, Any]]] = mapped_column(
11301     JSONB,
11302     nullable=True,
11303 )
11304 created_at: Mapped[datetime] = mapped_column(
11305     DateTime(timezone=True),
11306     nullable=False,
11307     server_default=func.now(),
11308 )
11309 // File: app/db/models/plan.py
11310 from __future__ import annotations
11311
11312 from datetime import datetime
11313 from typing import Any, Optional
11314 from uuid import UUID, uuid4
11315
11316 from sqlalchemy import CheckConstraint, DateTime, ForeignKey, Integer, String, Text, Uuid,
func, text
11317 from sqlalchemy.dialects.postgresql import JSONB
11318 from sqlalchemy.orm import Mapped, mapped_column
11319
11320 from app.db.base import Base
11321
11322
11323 class PlanRequestRecord(Base):
11324     __tablename__ = "plan_requests"
11325     __table_args__ = (
11326         CheckConstraint(
11327             "status in ('pending', 'succeeded', 'failed')",
11328             name="plan_requests_status_check",
11329         ),
11330         CheckConstraint(
11331             "intent in ('daily', 'occasion', 'diy', 'size_adjustment')",
11332             name="plan_requests_intent_check",
11333         ),
11334     )
11335
11336 id: Mapped[UUID] = mapped_column(Uuid(as_uuid=True), primary_key=True, default=uuid4)
11337 analysis_id: Mapped[Optional[str]] = mapped_column(
11338     String(128),

```

```

11339         ForeignKey("analysis_requests.analysis_id", ondelete="SET NULL"),
11340         nullable=True,
11341     )
11342     intent: Mapped[str] = mapped_column(String(32), nullable=False)
11343     user_preferences: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
11344     provider_name: Mapped[str] = mapped_column(String(64), nullable=False)
11345     provider_model: Mapped[str] = mapped_column(String(128), nullable=False)
11346     status: Mapped[str] = mapped_column(String(32), nullable=False)
11347     error_code: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
11348     error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
11349     latency_ms: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
11350     request_received_at: Mapped[datetime] = mapped_column(
11351         DateTime(timezone=True),
11352         nullable=False,
11353         server_default=func.now(),
11354     )
11355
11356
11357 class PlanResultRecord(Base):
11358     __tablename__ = "plan_results"
11359     __table_args__ = (
11360         CheckConstraint(
11361             "difficulty in ('easy', 'medium', 'hard')",
11362             name="plan_results_difficulty_check",
11363         ),
11364         CheckConstraint("ordinal >= 0", name="plan_results_ordinal_check"),
11365     )
11366
11367     id: Mapped[UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid4)
11368     plan_request_id: Mapped[UUID] = mapped_column(
11369         UUID(as_uuid=True),
11370         ForeignKey("plan_requests.id", ondelete="CASCADE"),
11371         nullable=False,
11372     )
11373     ordinal: Mapped[int] = mapped_column(Integer, nullable=False)
11374     title: Mapped[str] = mapped_column(String(256), nullable=False)
11375     summary: Mapped[str] = mapped_column(Text, nullable=False)
11376     difficulty: Mapped[str] = mapped_column(String(16), nullable=False)
11377     materials_json: Mapped[list[str]] = mapped_column(
11378         JSONB,
11379         nullable=False,
11380         server_default=text("'[]'::jsonb"),
11381     )
11382     estimated_time: Mapped[str] = mapped_column(String(128), nullable=False)
11383     steps_json: Mapped[list[dict[str, str]]] = mapped_column(
11384         JSONB,
11385         nullable=False,
11386         server_default=text("'[]'::jsonb"),
11387     )
11388     reasoning_note: Mapped[Optional[str]] = mapped_column(Text, nullable=True)

```

```

11389     raw_provider_payload_json: Mapped[Optional[dict[str, Any]]] = mapped_column(
11390         JSONB,
11391         nullable=True,
11392     )
11393     plan_id: Mapped[str] = mapped_column(
11394         String(128),
11395         unique=True,
11396         index=True,
11397         nullable=False,
11398         default=lambda: f"plan-{uuid4().hex}",
11399     )
11400     created_at: Mapped[datetime] = mapped_column(
11401         DateTime(timezone=True),
11402         nullable=False,
11403         server_default=func.now(),
11404     )
11405 // File: app/db/models/preview.py
11406 from __future__ import annotations
11407
11408 from datetime import datetime
11409 from enum import Enum
11410 from typing import Optional
11411 from uuid import UUID, uuid4
11412
11413 from sqlalchemy import (
11414     BigInteger,
11415     CheckConstraint,
11416     DateTime,
11417     ForeignKey,
11418     Integer,
11419     String,
11420     Text,
11421     UniqueConstraint,
11422     Uuid,
11423     func,
11424     text,
11425 )
11426 from sqlalchemy.dialects.postgresql import JSONB
11427 from sqlalchemy.orm import Mapped, mapped_column
11428
11429 from app.db.base import Base
11430
11431
11432 class PreviewRenderMode(str, Enum):
11433     SIMULATION = "simulation"
11434
11435
11436 class PreviewJobStatus(str, Enum):
11437     QUEUED = "queued"
11438     RUNNING = "running"

```

```

11439     COMPLETED = "completed"
11440     COMPLETED_WITH_FAILURES = "completed_with_failures"
11441     FAILED = "failed"
11442     EXPIRED = "expired"
11443
11444
11445     class PreviewPlanStatus(str, Enum):
11446         QUEUED = "queued"
11447         RUNNING = "running"
11448         COMPLETED = "completed"
11449         FAILED = "failed"
11450         FILTERED = "filtered"
11451
11452
11453     class PreviewAssetKind(str, Enum):
11454         BEFORE = "before"
11455         AFTER = "after"
11456         COMPARISON = "comparison"
11457
11458
11459     class PreviewJobRecord(Base):
11460         __tablename__ = "preview_jobs"
11461         __table_args__ = (
11462             CheckConstraint(
11463                 "status in ('queued', 'running', 'completed', 'completed_with_failures',
11464                 'failed', 'expired')",
11465                 name="preview_jobs_status_check",
11466             ),
11467             CheckConstraint(
11468                 "render_mode in ('simulation')",
11469                 name="preview_jobs_render_mode_check",
11470             ),
11471         )
11472         preview_job_id: Mapped[str] = mapped_column(
11473             String(128),
11474             primary_key=True,
11475             default=lambda: f"preview-job-{uuid4().hex}",
11476         )
11477         plan_request_id: Mapped[UUID] = mapped_column(
11478             Uuid(as_uuid=True),
11479             ForeignKey("plan_requests.id", ondelete="CASCADE"),
11480             nullable=False,
11481         )
11482         render_mode: Mapped[str] = mapped_column(
11483             String(32),
11484             nullable=False,
11485             server_default=text("'simulation'"),
11486             default=PreviewRenderMode.SIMULATION.value,
11487         )
11488         status: Mapped[str] = mapped_column(

```



```

11489         String(40),
11490         nullable=False,
11491         server_default=text("'queued'"),
11492         default=PreviewJobStatus.QUEUED.value,
11493     )
11494     requested_plan_ids_json: Mapped[list[str]] = mapped_column(
11495         JSONB,
11496         nullable=False,
11497         server_default=text("'[]'::jsonb"),
11498         default=list,
11499     )
11500     tuning_json: Mapped[Optional[dict[str, str | None]]] = mapped_column(
11501         JSONB,
11502         nullable=True,
11503     )
11504     attempt_count: Mapped[int] = mapped_column(
11505         Integer,
11506         nullable=False,
11507         server_default=text("0"),
11508         default=0,
11509     )
11510     queued_at: Mapped[datetime] = mapped_column(
11511         DateTime(timezone=True),
11512         nullable=False,
11513         server_default=func.now(),
11514     )
11515     started_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True),
11516 nullable=True)
11517     completed_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True),
11518 nullable=True)
11519     expires_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True),
11520 nullable=True)
11521     error_code: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
11522     error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
11523     created_at: Mapped[datetime] = mapped_column(
11524         DateTime(timezone=True),
11525         nullable=False,
11526         server_default=func.now(),
11527     )
11528     updated_at: Mapped[datetime] = mapped_column(
11529         DateTime(timezone=True),
11530         nullable=False,
11531         server_default=func.now(),
11532         onupdate=func.now(),
11533     )
11534
11535 class PreviewResultRecord(Base):
11536     __tablename__ = "preview_results"
11537     __table_args__ = (
11538         CheckConstraint(
11539             "status in ('queued', 'running', 'completed', 'failed', 'filtered')",
11540             name="preview_results_status_check",
11541         ),
11542     )

```

```

11539     ),
11540     CheckConstraint("ordinal >= 0", name="preview_results_ordinal_check"),
11541     UniqueConstraint("preview_job_id", "plan_id", name="uq_preview_results_job_plan_id"),
11542     UniqueConstraint("preview_job_id", "ordinal", name="uq_preview_results_job_ordinal"),
11543 )
11544
11545 id: Mapped[UUID] = mapped_column(
11546     Uuid(as_uuid=True),
11547     primary_key=True,
11548     default=uuid4,
11549 )
11550 preview_job_id: Mapped[str] = mapped_column(
11551     String(128),
11552     ForeignKey("preview_jobs.preview_job_id", ondelete="CASCADE"),
11553     nullable=False,
11554 )
11555 plan_id: Mapped[str] = mapped_column(
11556     String(128),
11557     ForeignKey("plan_results.plan_id", ondelete="CASCADE"),
11558     nullable=False,
11559 )
11560 ordinal: Mapped[int] = mapped_column(Integer, nullable=False)
11561 status: Mapped[str] = mapped_column(
11562     String(24),
11563     nullable=False,
11564     server_default=text("'queued'"),
11565     default=PreviewPlanStatus.QUEUED.value,
11566 )
11567 filter_reason: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
11568 error_code: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
11569 error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
11570 queued_at: Mapped[datetime] = mapped_column(
11571     DateTime(timezone=True),
11572     nullable=False,
11573     server_default=func.now(),
11574 )
11575 started_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True),
11576     nullable=True)
11577 completed_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True),
11578     nullable=True)
11579 created_at: Mapped[datetime] = mapped_column(
11580     DateTime(timezone=True),
11581     nullable=False,
11582     server_default=func.now(),
11583 )
11584
11584 class PreviewAssetRecord(Base):
11585     __tablename__ = "preview_assets"
11586     __table_args__ = (
11587         CheckConstraint(
11588             "asset_kind in ('before', 'after', 'comparison')",

```

```

11589         name="preview_assets_asset_kind_check",
11590     ),
11591     UniqueConstraint("preview_result_id", "asset_kind",
name="uq_preview_assets_result_kind"),
11592 )
11593
11594     id: Mapped[UUID] = mapped_column(
11595         Uuid(as_uuid=True),
11596         primary_key=True,
11597         default=uuid4,
11598     )
11599     preview_result_id: Mapped[UUID] = mapped_column(
11600         Uuid(as_uuid=True),
11601         ForeignKey("preview_results.id", ondelete="CASCADE"),
11602         nullable=False,
11603     )
11604     asset_kind: Mapped[str] = mapped_column(String(32), nullable=False)
11605     object_key: Mapped[str] = mapped_column(Text, nullable=False, unique=True)
11606     content_type: Mapped[Optional[str]] = mapped_column(String(128), nullable=True)
11607     byte_size: Mapped[Optional[int]] = mapped_column(BigInteger, nullable=True)
11608     checksum_sha256: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
11609     created_at: Mapped[datetime] = mapped_column(
11610         DateTime(timezone=True),
11611         nullable=False,
11612         server_default=func.now(),
11613     )
11614 // File: app/db/models/user.py
11615 from __future__ import annotations
11616
11617 from datetime import datetime
11618 from uuid import UUID, uuid4
11619
11620 from sqlalchemy import BigInteger, DateTime, ForeignKey, String, Text, Uuid, func, text
11621 from sqlalchemy.dialects.postgresql import JSONB
11622 from sqlalchemy.orm import Mapped, mapped_column
11623
11624 from app.db.base import Base
11625
11626
11627 class AppUserRecord(Base):
11628     __tablename__ = "app_users"
11629
11630     id: Mapped[UUID] = mapped_column(Uuid(as_uuid=True), primary_key=True, default=uuid4)
11631     clerk_user_id: Mapped[str] = mapped_column(String(128), unique=True, nullable=False,
index=True)
11632     primary_email: Mapped[str | None] = mapped_column(String(320), nullable=True)
11633     display_name: Mapped[str | None] = mapped_column(String(255), nullable=True)
11634     avatar_url: Mapped[str | None] = mapped_column(Text, nullable=True)
11635     created_at: Mapped[datetime] = mapped_column(
11636         DateTime(timezone=True),
11637         nullable=False,
11638         server_default=func.now(),

```

```

11639     )
11640     updated_at: Mapped[datetime] = mapped_column(
11641         DateTime(timezone=True),
11642         nullable=False,
11643         server_default=func.now(),
11644         onupdate=func.now(),
11645     )
11646     last_seen_at: Mapped[datetime] = mapped_column(
11647         DateTime(timezone=True),
11648         nullable=False,
11649         server_default=func.now(),
11650     )
11651
11652
11653 class UserHistorySnapshotRecord(Base):
11654     __tablename__ = "user_history_snapshots"
11655
11656     id: Mapped[UUID] = mapped_column(Uuid(as_uuid=True), primary_key=True, default=uuid4)
11657     user_id: Mapped[UUID] = mapped_column(
11658         Uuid(as_uuid=True),
11659         ForeignKey("app_users.id", ondelete="CASCADE"),
11660         nullable=False,
11661         unique=True,
11662         index=True,
11663     )
11664     schema_version: Mapped[int] = mapped_column(
11665         BigInteger,
11666         nullable=False,
11667         server_default=text("1"),
11668         default=1,
11669     )
11670     snapshot_json: Mapped[dict[str, object]] = mapped_column(
11671         JSONB,
11672         nullable=False,
11673         server_default=text("'{}'::jsonb"),
11674         default=dict,
11675     )
11676     revision: Mapped[int] = mapped_column(
11677         BigInteger,
11678         nullable=False,
11679         server_default=text("0"),
11680         default=0,
11681     )
11682     bootstrapped_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True),
11683 nullable=True)
11683     updated_at: Mapped[datetime] = mapped_column(
11684         DateTime(timezone=True),
11685         nullable=False,
11686         server_default=func.now(),
11687         onupdate=func.now(),
11688     )

```